Contents

1 Documentation technique - SAE Application intelligente **9**
1.1 Table des matières 9

2 Accueil **9**

3 Gitelligence - Application intelligente **9**
3.1 Version PDF 10
3.2 Documentation technique 10
3.3 CI et déploiement 10

4 Présentation **10**

5 Présentation du code source **10**
5.1 Objet 10
5.2 Organisation 10
5.3 Structure principale 10
5.4 Rôle de cette documentation 11
5.5 À retenir 11

6 Point d'entrée **11**

7 Point d'Entrée Principal CLI **11**
7.1 Description 11
7.2 Responsabilités 11
7.3 Arguments de la ligne de commande 11
7.4 Points clés 12
7.5 Rôle dans l'architecture 12
7.6 Détails déplacés du code source 12
7.7 Commentaires de fonction (dupliqués du code source) 12

8 Module Go **12**

9 Module Go CLI **12**
9.1 Description 12
9.2 Responsabilités 12
9.3 Points clés 13
9.4 Rôle dans l'architecture 13

10 Client **13**

11 Interface BackendClient et Coordinateur BridgeClient **13**
11.1 Description 13
11.2 Responsabilités 13
11.3 Points clés 13
11.4 Détails déplacés du code source 13
11.5 Commentaires de fonction (dupliqués du code source) 14

12 Backend **14**

13 Client de Pontage Go ↔ Python (Bridge) **14**
13.1 Description 14
13.2 Architecture & Responsabilités 15
13.3 Points clés 15
13.4 Rôle dans l'architecture 15

14 Environnement **16**

15 Gestionnaire d'Environnement (.env) **16**
15.1 Description . . . 16
15.2 Responsabilités . . . 16
15.3 Points clés . . . 16
15.4 Détails déplacés du code source . . . 16
15.5 Commentaires de fonction (dupliqués du code source) . . . 17

16 Mock **17**

17 Implémentation Simulée MockClient **17**
17.1 Description . . . 17
17.2 Responsabilités . . . 17
17.3 Points clés . . . 17
17.4 Détails déplacés du code source . . . 18
17.5 Commentaires de fonction (dupliqués du code source) . . . 18

18 Client Python **18**

19 Implémentation Réelle PythonClient **18**
19.1 Description . . . 18
19.2 Responsabilités . . . 19
19.3 Points clés . . . 19
19.4 Détails déplacés du code source . . . 19
19.5 Commentaires de fonction (dupliqués du code source) . . . 19

20 Modèles **20**

21 Modèles de Données Typées CLI **20**
21.1 Description . . . 20
21.2 Structures principales . . . 20
21.2.1 RepoStatus . . . 20
21.2.2 DiffFile . . . 20
21.2.3 CommitProposal . . . 20
21.2.4 ConfigResponse & PingResponse . . . 20
21.3 Rôle dans l'architecture . . . 21
21.4 Détails déplacés du code source . . . 21

22 Boîtes **21**

23 Composants de Boîtes et Tableaux UI **21**
23.1 Description . . . 21
23.2 Responsabilités . . . 21
23.3 Points clés . . . 22
23.4 Rôle dans l'architecture . . . 22
23.5 Commentaires de fonction (dupliqués du code source) . . . 22

24 Spinner **22**

25 Animation Spinner et Gestion Asynchrone **22**
25.1 Description . . . 22
25.2 Responsabilités . . . 22
25.3 Points clés . . . 23
25.4 Rôle dans l'architecture . . . 23
25.5 Commentaires de fonction (dupliqués du code source) . . . 23

26 Styles **23**

27 Moteur de Styles et Séquences ANSI **23**
27.1 Description . . . 23
27.2 Responsabilités . . . 23
27.3 Points clés . . . 24
27.4 Rôle dans l'architecture . . . 24
27.5 Commentaires de fonction (dupliqués du code source) . . . 24

28 Menu **24**

29 Vue Menu Principal **24**
29.1 Description . . . 24
29.2 Responsabilités . . . 24
29.3 Points clés . . . 25
29.4 Rôle dans l'architecture . . . 25
29.5 Commentaires de fonction (dupliqués du code source) . . . 25

30 Commit **25**

31 Vue Génération de Commit **25**
31.1 Description . . . 25
31.2 Workflow en 4 étapes . . . 25
31.3 Points clés . . . 26
31.4 Rôle dans l'architecture . . . 26
31.5 Détails déplacés du code source . . . 26
31.6 Commentaires de fonction (dupliqués du code source) . . . 26

32 Config **26**

33 Vue Configuration **26**
33.1 Description . . . 26
33.2 Responsabilités . . . 26
33.3 Points clés . . . 27
33.4 Rôle dans l'architecture . . . 27
33.5 Commentaires de fonction (dupliqués du code source) . . . 27

34 Release notes **27**

35 Vue Release Notes **27**
35.1 Description . . . 27
35.2 Responsabilités . . . 27
35.3 Points clés . . . 28
35.4 Rôle dans l'architecture . . . 28
35.5 Détails déplacés du code source . . . 28
35.6 Commentaires de fonction (dupliqués du code source) . . . 28

36 init **28**

37 Module git_commit_release_notes_generator **28**
37.1 Rôle . . . 28
37.2 Sous-modules . . . 28
37.3 Objectif fonctionnel . . . 28

38 config **29**

39 Configuration **29**
39.1 Description . . . 29
39.2 Variables principales . . . 29
39.3 Emplacement et chargement du fichier .env . . . 29
39.4 Points de vigilance . . . 29
39.5 Exemple d'usage . . . 29
39.6 Détails déplacés du code source . . . 30
39.7 Commentaires de fonction (dupliqués du code source) . . . 30

40 models **30**

41 Modèles de données **30**
41.1 Description . . . 30
41.2 Structures principales . . . 30
41.2.1 DiffFile . . . 30
41.2.2 CommitMessage . . . 30
41.3 Rôle dans le flux . . . 31
41.4 Détails déplacés du code source . . . 31
41.5 Commentaires de fonction (dupliqués du code source) . . . 31

42 init **31**

43 Package ollama_client **31**
43.1 Rôle . . . 31
43.2 Sous-modules . . . 31
43.3 Objectif . . . 32

44 base **32**

45 Base Ollama client **32**
45.1 Description . . . 32
45.2 Rôle attendu . . . 32
45.3 Utilisation . . . 32

46 embedding **32**

47 Embedding Ollama **32**
47.1 Description . . . 32
47.2 Statut actuel . . . 32
47.3 Rôle fonctionnel attendu . . . 33

48 exceptions **33**

49 Exceptions Ollama **33**
49.1 Description . . . 33
49.2 Exceptions prévues . . . 33
49.3 Pourquoi les centraliser ? . . . 33

50 llm **33**

51 Client LLM **33**
51.1 Description . . . 33
51.2 Flux de travail . . . 34
51.3 Points clés . . . 34
51.4 Fonctions exposées . . . 34
51.5 Dépendances . . . 34

51.6 Détails déplacés du code source . . . 35
51.7 Commentaires de fonction (dupliqués du code source) . . . 35

52 mock_llm **36**

53 Module de Simulation de l'IA (Mock LLM) **36**
53.1 Description . . . 36
53.2 Responsabilités . . . 36
53.3 Fonctions exposées . . . 36
53.4 Points clés . . . 36
53.5 Commentaires de fonction (dupliqués du code source) . . . 37

54 Prompt système **37**

55 Prompt système de génération de commit **37**
55.1 Description . . . 37
55.2 Objectif . . . 37
55.3 Rôle dans le flux . . . 37

56 init **37**

57 Service du générateur **37**
57.1 Rôle . . . 37
57.2 Sous-modules . . . 37
57.3 Objectif . . . 38
57.4 Détails déplacés du code source . . . 38

58 core **38**

59 Core du service (Point d'entrée du pont Go ↔ Python) **38**
59.1 Description . . . 38
59.2 Contrat d'interface (Règle d'or Go / Python) . . . 38
59.3 Les 9 actions gérées (`--action <NOM>`) . . . 38
59.4 Fonctions clés . . . 39
59.5 Détails déplacés du code source . . . 39
59.6 Commentaires de fonction (dupliqués du code source) . . . 40

60 git_wrapper **40**

61 Wrapper Git **40**
61.1 Description . . . 40
61.2 Responsabilités . . . 40
61.3 Points clés . . . 41
61.3.1 `GitWrapper` . . . 41
61.3.2 `GitWrapperError` . . . 41
61.3.3 `CommitInfo` . . . 41
61.4 Rôle dans le flux . . . 41
61.5 Détails déplacés du code source . . . 41
61.6 Commentaires de fonction (dupliqués du code source) . . . 42

62 utils **42**

63 Utilitaires de service **42**
63.1 Description . . . 42
63.2 Rôle attendu . . . 42
63.3 À compléter . . . 42

64 Métriques **43**

65 Script de mesure de conformité et de latence **43**
65.1 Description . . . 43
65.2 Flux de travail . . . 43
65.3 Points clés . . . 43
65.4 Dépendances . . . 43
65.5 Détails déplacés du code source . . . 43
65.6 Commentaires de fonction (dupliqués du code source) . . . 43

66 PDF **44**

67 Script de génération du PDF de documentation **44**
67.1 Description . . . 44
67.2 Flux de travail . . . 44
67.3 Points clés . . . 44
67.4 Commentaires de fonction (dupliqués du code source) . . . 44

68 CLI Go (guide) **44**

69 Architecture et Fonctionnement de l'Interface CLI (Go) **44**
69.1 Guide Pratique : Compiler, Tester et Présenter . . . 44
69.1.1 Compiler le projet : . . . 44
69.1.2 Options de la ligne de commande (Flags) . . . 45
69.1.3 Lancer l'application : . . . 46
69.1.4 Exécuter les tests unitaires Go : . . . 46
69.2 Pourquoi avoir choisi Go pour le Front-end ? . . . 46
69.3 Comment Go communique avec Python (Le Pont / Bridge) . . . 46
69.3.1 Le mécanisme d'appel système (`exec.Command`) . . . 46
69.3.2 Le protocole JSON et la règle de `stdout` vs `stderr` . . . 47
69.3.3 Architecture du Mock & Inversion des Dépendances (DIP) . . . 47
69.4 Architecture Interne du Code Go (`src/cli/`) . . . 48
69.4.1 Arborescence détaillée . . . 48
69.4.2 Rôle de chaque paquet : . . . 49
69.5 Fonctionnement Écran par Écran . . . 49
69.5.1 Écran 1 : Menu Principal & Contexte Dépôt . . . 49
69.5.2 Écran 2 : Génération de Commit & Boucle de Décision . . . 49
69.5.3 Écran 3 : Génération des Release Notes . . . 50
69.5.4 Écran 4 : Configuration (.env) & Diagnostic Ping Ollama . . . 50

70 Présentation **51**

71 Présentation des tests **51**
71.1 Objet . . . 51
71.2 Types de tests présents . . . 51
71.3 Organisation . . . 51
71.4 Rôle de cette documentation . . . 51
71.5 À retenir . . . 51

72 Core (point d'entrée) **51**

73 Tests unitaires du point d'entrée Core **51**
73.1 Description . . . 51
73.2 Cas de tests couverts . . . 52

74 Git wrapper **52**

75 Tests du wrapper Git **52**
75.1 Description . . . 52
75.2 Ce qui est testé . . . 52
75.3 Cas de test principaux . . . 53
75.3.1 1. Parsing d'un diff multi-fichiers . . . 53
75.3.2 2. Sanitisation des secrets . . . 53
75.3.3 3. Structure du payload JSON . . . 53
75.4 Points de vigilance . . . 53

76 Intégration Git wrapper (slow) **53**

77 Test d'intégration réel avec le wrapper Git **53**
77.1 Description . . . 53
77.2 Ce qui est testé . . . 54
77.3 Stratégie de test . . . 54
77.4 Cas importants . . . 54
77.4.1 compatibilité de contrat entre deux modules . . . 54
77.5 Points de vigilance . . . 54

78 Ollama LLM **54**

79 Tests du client Ollama **54**
79.1 Description . . . 54
79.2 Ce qui est testé . . . 54
79.3 Stratégie de test . . . 55
79.4 Cas importants . . . 55
79.4.1 génération réussie au premier essai . . . 55
79.4.2 retry puis succès . . . 55
79.4.3 validation stricte du message . . . 55
79.5 Points de vigilance . . . 55

80 Ollama LLM (slow, réseau réel) **56**

81 Test réseau réel du client Ollama **56**
81.1 Description . . . 56
81.2 Ce qui est testé . . . 56
81.3 Stratégie de test . . . 56
81.4 Cas importants . . . 56
81.4.1 génération contre le vrai modèle . . . 56
81.5 Points de vigilance . . . 56

82 Mock IA **56**

83 Tests du Mock IA (Simulation Hors-Ligne) **56**
83.1 Description . . . 56
83.2 Ce qui est testé . . . 57
83.3 Stratégie de test . . . 57
83.4 Exécution des tests . . . 57

84 Golden diffs **57**

85 Fixtures Golden Diff **57**
85.1 Description . . . 58
85.2 Structure de données . . . 58
85.3 Rôle fonctionnel . . . 58
85.4 Exemple de scénarios . . . 58

85.5 Points de vigilance . . . 58

86 Golden diffs (slow, réseau réel) 59

87 Test réseau réel sur les golden files 59
87.1 Description . . . 59
87.2 Ce qui est testé . . . 59
87.3 Stratégie de test . . . 59
87.4 Cas importants . . . 59
87.4.1 comparaison type attendu / type obtenu . . . 59
87.5 Points de vigilance . . . 59

88 Bridge 59

89 Tests Unitaires du Bridge Go 59
89.1 Description . . . 59
89.2 Tests couverts . . . 60
89.3 Exécution des tests . . . 60

90 Module Go 60

91 Module Go de Test 60
91.1 Description . . . 60
91.2 Responsabilités . . . 60

92 Initialisation 61

93 Activation des situations de test 61
93.1 Description . . . 61
93.2 Rôle fonctionnel . . . 61
93.3 Comportement . . . 61
93.4 Utilité dans le projet . . . 61

94 Désinitialisation 61

95 Neutralisation des situations de test 61
95.1 Description . . . 61
95.2 Rôle fonctionnel . . . 62
95.3 Objectif . . . 62
95.4 Comportement . . . 62
95.5 Utilité du projet . . . 62

96 Déploiement de la documentation 62

97 Déploiement de la documentation 62
97.1 Objectif . . . 62
97.2 Fichiers concernés . . . 62
97.3 Configuration MkDocs . . . 63
97.4 Prérequis : activation manuelle de GitHub Pages (une seule fois) . . . 63
97.5 Prérequis du PDF versionné . . . 63
97.6 Workflow de déploiement . . . 63
97.6.1 Déclenchement . . . 63
97.6.2 Marqueur de version de développement . . . 63
97.6.3 Étapes du job `build` . . . 64
97.6.4 Étapes du job `deploy` . . . 64
97.7 Résultat attendu . . . 64

97.8 Bonnes pratiques . . . 64
97.9 Commande de vérification locale . . . 64

98 Tests automatiques via GitHub Actions 64

99 Tests automatisés via GitHub Actions 64
99.1 Objectif . . . 65
99.2 Workflows disponibles . . . 65
99.3 Workflow CI . . . 65
99.3.1 Jobs . . . 65
99.4 Vérification locale . . . 65
99.5 Intérêt du workflow CI . . . 66

100 Sprint 00 66

101 Sprint N°1 : Mise en place du projet 66

102 Sprint 01 66

103 Sprint N°2 : Intégration et développement des fonctionnalités 66

104 Sprint 02 67

105 Sprint N°3 : Intégration backend/CLI et fiabilisation du client LLM 67

106 Sprint 03 68

107 Sprint N°4 : Simulation hors-ligne (Mock IA), robustesse backend et déploiement continu de la documentation 68

108 Sprint 04 69

109 Sprint N°5 : Diagnostic réseau unifié, stabilisation CI/CD, version stable sur main et améliorations UX 69

1 Documentation technique - SAE Application intelligente

1.1 Table des matières

Contents

---

2 Accueil

3 Gitelligence - Application intelligente

Documentation du projet Gitelligence (application intelligente).

Cette documentation présente l'architecture technique, les tests, le suivi de projet et les mécanismes de CI/CD associés.

3.1 Version PDF

La documentation est également disponible en version PDF téléchargeable, régénérée automatiquement à chaque commit.

3.2 Documentation technique

La documentation technique suit la structure du code source et des tests.

- `doc_src_files` : documentation des fichiers du dossier `src`
- `doc_test_files` : documentation des fichiers du dossier `test`
- `monitoring` : suivi des sprints et de l'avancement du projet

3.3 CI et déploiement

Le projet met en place deux éléments essentiels de documentation opérationnelle :

- le déploiement de la documentation : explication du build MkDocs et de la publication sur GitHub Pages
- les tests automatiques via GitHub Actions : explication du lancement des tests et de leur organisation dans la CI

---

4 Présentation

5 Présentation du code source

Cette section regroupe la documentation technique du code applicatif du projet.

5.1 Objet

Elle permet de suivre la structure du dossier `src/` et de comprendre le rôle de chaque module, en gardant une correspondance claire entre le code et sa documentation.

5.2 Organisation

Le dossier `src/` contient le cœur applicatif du projet, notamment :

- la génération des messages de commit
- l'intégration avec Ollama
- la gestion Git et des diff
- les composants CLI
- les services de synthèse et d'analyse

5.3 Structure principale

- `git_commit_release_notes_generator/` : logique métier du projet
- `ollama_client/` : intégration avec les modèles LLM
- `service/` : services de traitement, Git et utilitaires
- `cli/` : interface en ligne de commande et vues utilisateur

5.4 Rôle de cette documentation

Chaque fichier de code a un équivalent Markdown dans ce dossier, avec la même organisation arborescente. Cela permet de :

- retrouver rapidement la description d'un module
- comprendre les responsabilités de chaque composant
- maintenir la documentation synchronisée avec l'évolution du code

5.5 À retenir

La documentation du code source doit rester alignée sur l'implémentation réelle. Quand un module est ajouté, renommé ou modifié, la documentation associée doit être mise à jour dans ce dossier.

---

6 Point d'entrée

7 Point d'Entrée Principal CLI

Fichier source associé : src/cli/main.go

7.1 Description

Ce fichier constitue le point d'entrée exécutable de l'interface en ligne de commande. Il initialise l'environnement, configure l'interception des signaux système, instancie le client de pontage vers Python et pilote la boucle d'événements principale du menu.

7.2 Responsabilités

- analyser les arguments et flags de la ligne de commande (`--dev`, `--mockInterface`, `--mockIA`, `--repo`, `--app-dir`) ;
- localiser la racine applicative via `FindAppRoot` pour pointer sur le `.env` de l'application indépendamment du dépôt cible analysé ;
- intercepter les signaux d'interruption système (`SIGINT` / `SIGTERM` / `Ctrl+C`) pour restaurer l'état du terminal (réaffichage du curseur et réinitialisation des couleurs) ;
- initialiser le `BackendClient` relié au répertoire du dépôt Git et à la racine de l'application (en respectant la priorité du fichier `.env`) ;
- interroger l'état du dépôt auprès du backend Python à chaque itération ;
- aiguiller l'utilisateur vers les différentes vues (`commit`, `release-notes`, `config`, ou sortie propre) en transmettant le mode développeur `--dev` à l'écran de configuration.

7.3 Arguments de la ligne de commande

- `--repo <chemin>` : chemin vers le dépôt Git cible à analyser (défaut : `.`).
- `--dev` : active le mode développeur, autorisant l'utilisation des drapeaux de simulation en CLI et la modification des réglages de mock dans l'écran de configuration.
- `--mockInterface` : activation forcée du mock de l'interface (`MockClient`) sans exécuter le backend Python (requiert obligatoirement `--dev`).
- `--mockIA` : activation forcée du mock de l'IA sans serveur Ollama (requiert obligatoirement `--dev`).

- `--app-dir <chemin>` : chemin explicite vers la racine de l'application Gitelligence (où se trouvent le code et le `.env` applicatif). Permet d'isoler à 100% la configuration applicative lorsque le binaire est exécuté sur un dépôt tiers.

7.4 Points clés

- **Gestion des signaux OS** : mise en place d'une goroutine dédiée et d'un `signal.Notify` pour intercepter `Ctrl+C` et garantir que le terminal ne reste jamais dans un état dégradé (curseur masqué) ;
- **Boucle interactive robuste** : gestion de la fin de flux (EOF) et réaffichage automatique du contexte après chaque action ;
- **Séparation stricte dépôt cible vs application** : l'application dissocie totalement le dépôt analysé (`repoRoot`) de l'arborescence de l'outil (`appRoot`), empêchant toute écriture de `.env` dans le projet inspecté.

7.5 Rôle dans l'architecture

`main.go` est le chef d'orchestre du front-end. Il lie la couche de présentation (vues et utilitaires ANSI) au client de pontage backend sans intégrer la moindre règle métier.

7.6 Détails déplacés du code source

Précisions issues des commentaires retirés de `main.go` et non redondantes avec les sections ci-dessus :

- En cas d'erreur de `GetRepoStatus`, le statut est dégradé à `nil` (`status = nil`) et le menu principal est quand même affiché via `views.ShowMainMenu`.
- Toute option non reconnue dans le `switch` (`default`) provoque un retour direct au menu, sans message d'erreur ni sortie.

7.7 Commentaires de fonction (dupliqués du code source)

- `main()` : Point d'entrée initialisant l'interface et pilotant le menu principal.

---

8 Module Go

9 Module Go CLI

Fichier source associé : src/cli/go.mod

9.1 Description

Ce fichier définit le module Go standard (`sae-git-cli`) utilisé pour compiler l'interface terminal (TUI / CLI) de l'application.

9.2 Responsabilités

- déclarer le nom canonique du module Go (`sae-git-cli`) ;
- fixer la version minimale requise du compilateur Go (`go 1.22`) ;
- garantir l'autonomie totale du module sans dépendances tierces externes.

9.3 Points clés

- **Zéro dépendance externe** : l'interface utilise exclusivement les packages de la bibliothèque standard Go (`fmt`, `os`, `os/exec`, `bufio`, `encoding/json`, `time`, `os/signal`, `regexp`, `math`) ;
- compilation instantanée et portabilité native sur tout environnement Linux / macOS sans téléchargement via proxy ou connexion internet.

9.4 Rôle dans l'architecture

Point d'ancrage pour le gestionnaire de paquets et l'outil de build Go (`go build`, `go run`).

---

10 Client

11 Interface BackendClient et Coordinateur BridgeClient

Fichier source associé : src/cli/bridge/client.go

11.1 Description

Ce fichier définit l'interface maîtresse `BackendClient` et le coordinateur adaptateur `BridgeClient` qui fait le lien entre les vues de l'interface en ligne de commande et les implémentations réelles (`PythonClient`) ou simulées (`MockClient`).

11.2 Responsabilités

- Définir le contrat d'interface haut niveau `BackendClient` regroupant les actions du domaine Git, Ollama et configuration.
- Instancier le coordinateur via `NewBackendClientWithAppRoot(repoRoot, appRoot, forceMockInterface, forceMockIA)`.
- Maintenir la référence vers le dépôt cible (`repoRoot`) et la racine de l'application (`appRoot`).
- Assurer le routage dynamique vers le délégué actif en fonction du drapeau `--mockInterface` ou du paramètre `MOCK_INTERFACE` du `.env`.
- Fournir les méthodes de consultation et modification de la configuration (`GetConfig`, `SaveConfig`).

11.3 Points clés

- **Inversion des dépendances (DIP)** : Le front-end dépend exclusivement de l'abstraction `BackendClient`.
- **Zéro logique métier en Go** : Les traitements d'analyse Git, LLM et release notes sont délégués au backend Python.

11.4 Détails déplacés du code source

Précisions issues des commentaires retirés de `client.go` :

- `BridgeClient` coordonne l'accès au backend en déléguant à `PythonClient` ou `MockClient` ; `activeDelegate` renvoie l'implémentation active selon `IsMock`.

- IsMock indique si le mode simulation est actif : le drapeau forcé forceMockInterface est prioritaire, sinon lecture via ReadMockSettingFromEnv.
- GetConfig lit la configuration depuis le .env de l'application et force MockAI à true si forceMockIA est actif.
- SaveConfig persiste la nouvelle configuration dans le .env applicatif puis actualise l'état du client : mise à jour dynamique de isMock sauf si forcé par --mockInterface, et notification du backend Python via save-config uniquement en mode réel.

11.5 Commentaires de fonction (dupliqués du code source)

- NewBackendClient() : Instancie le client backend en résolvant automatiquement la racine applicative.
- NewBackendClientWithAppRoot() : Instancie le coordinateur avec dépôt cible et racine applicative explicites.
- BridgeClient.activeDelegate() : Sélectionne le délégué actif selon le mode simulation.
- BridgeClient.IsMock() : Indique si le mode simulation est actif.
- BridgeClient.GetRepoStatus() : Délègue la récupération du statut du dépôt.
- BridgeClient.GetDiff() : Délègue la récupération du diff.
- BridgeClient.StageAll() : Délègue l'indexation de tous les fichiers.
- BridgeClient.GenerateCommit() : Délègue la génération de la proposition de commit.
- BridgeClient.ApplyCommit() : Délègue l'application du message de commit.
- BridgeClient.Push() : Délègue la poussée vers le dépôt distant.
- BridgeClient.GetReleaseNotes() : Délègue la génération des notes de version.
- BridgeClient.PingOllama() : Délègue la vérification de joignabilité d'Ollama.
- BridgeClient.SaveMarkdownFile() : Délègue la sauvegarde du fichier Markdown.
- BridgeClient.GetConfig() : Charge la configuration depuis l'environnement applicatif.
- BridgeClient.SaveConfig() : Persiste la configuration et notifie le backend réel si nécessaire.

---

12 Backend

13 Client de Pontage Go ↔ Python (Bridge)

Fichiers sources associés : - src/cli/bridge/client.go : Interface publique BackendClient et coordinateur BridgeClient - src/cli/bridge/python_client.go : Implémentation réelle exécutant le sous-processus Python - src/cli/bridge/mock_client.go : Implémentation de test et mode démo hors-ligne - src/cli/bridge/env.go : Gestionnaire isolé du fichier .env (lecture, écriture, parsing) - test/cli/bridge_test.go : Tests unitaires automatisés du module bridge (situés dans test/cli/)

13.1 Description

Ce module implémente le pont de communication entre l'interface TUI Go et le backend Python. Il garantit la règle d'architecture **Zéro logique en Go** en s'appuyant sur le principe d'inversion des dépendances (*DIP*) : les vues front-end dépendent exclusivement de l'interface BackendClient.

13.2 Architecture & Responsabilités

1. **`BackendClient` (Interface)** :
   - Définit le contrat haut niveau et fortement typé (`GetRepoStatus`, `GetDiff`, `StageAll`, `GenerateCommit`, `ApplyCommit`, `Push`, `GetReleaseNotes`, `GetConfig`, `SaveConfig`, `PingOllama`, `SaveMarkdownFile`, `IsMock`).
2. **`PythonClient` (Mode Standard)** :
   - Exécute les commandes en sous-processus via `exec.Command` (`python3 -m git_commit_release_notes_generator.service.core`) ;
   - Capture `stdout` pour la désérialisation JSON stricte via `encoding/json` ;
   - Isole les traces et logs sur `stderr` pour ne pas corrompre le flux de données.
3. **`MockClient` (Mode Démo & Tests)** :
   - Fournit les réponses conformes aux jeux d'essais du projet (secrets masqués dans `auth/token_vault.py`, exclusion des fichiers binaires, simulation de délai) ;
   - Permet l'injection d'erreurs ou de réponses sur mesure (`CustomError`, `CustomDiff`, `SimulateDelay=false`) pour les tests unitaires automatisés.
4. **`BridgeClient` (Coordinateur Adaptateur)** :
   - Instancié par `NewBackendClientWithAppRoot(repoRoot, appRoot, forceMockInterface, forceMockIA)` ;
   - Achemine dynamiquement les appels vers `MockClient` ou `PythonClient` selon la variable `MOCK_INTERFACE` du `.env` ou le drapeau `--mockInterface` sans recompilation.
5. **`env.go` (Gestionnaire .env)** :
   - Localisation automatique de la racine de l'application (`FindAppRoot`) par exploration ascendante via `runtime.Caller`, l'exécutable Go (`os.Executable`) et le répertoire courant (`os.Getwd`) ;
   - Résolution stricte à la racine (`FindAppEnvPath`) : garantit que le fichier `.env` est toujours à la racine du projet (`SAE_sujet_8_application_intelligente/.env`), jamais dans `src/` ni dans le dépôt cible analysé (`repoRoot`) ;
   - Persistance sécurisée (`WriteEnvFile`) : écriture atomique du `.env` à la racine et suppression automatique de tout résidu dans `src/` ;
   - Tests automatisés (`test/cli/bridge_test.go`) : validation de l'isolation du dépôt cible (`TestAppEnvIsolationFromTargetRepo`) et interdiction stricte de tout fichier `.env` dans `src/` (`TestAppEnvNeverInSrc`).

13.3 Points clés

- **Découplage strict** : Le front-end dépend d'une abstraction (`BackendClient`), facilitant les tests unitaires isolés.
- **Résilience et isolation** : Séparation claire entre les appels système et l'environnement de test autonome.
- **Bascule à chaud** : Modification directe de `MOCK_INTERFACE` depuis l'écran de configuration du TUI répercutée immédiatement sans recompilation.

13.4 Rôle dans l'architecture

Passerelle unique d'accès au domaine métier. Toute vue front-end interagit avec le système exclusivement par l'intermédiaire de l'interface `BackendClient`.

---

14 Environnement

15 Gestionnaire d'Environnement (.env)

Fichier source associé : src/cli/bridge/env.go

15.1 Description

Ce module isole l'ensemble de la logique de localisation, de lecture et d'écriture du fichier de configuration `.env` de l'application Gitelligence.

15.2 Responsabilités

- **Localisation stricte de la racine applicative (`FindAppRoot`)** : exploration hiérarchique ascendante combinant `runtime.Caller`, `os.Executable` et `os.Getwd` pour déterminer l'emplacement physique du dépôt applicatif, indépendamment du dépôt Git cible analysé (`--repo`).
- **Garantie d'emplacement (`FindAppEnvPath`)** : renvoie l'emplacement unique `<racine_projet>/.env`. Élimine tout sous-chemin `src/` ou `cli/` pour empêcher la création ou lecture d'un `.env` dans les sous-dossiers.
- **Lecture typée (`LoadConfigFromEnv`)** : parse les variables `OLLAMA_BASE_URL`, `OLLAMA_MODEL`, `OLLAMA_TIMEOUT_S`, `MOCK_INTERFACE` et `MOCK_AI` avec valeurs par défaut de repli sécurisées.
- **Persistance atomique (`WriteEnvFile`)** : sauvegarde à chaud les réglages dans le `.env` à la racine de l'application (y compris `MOCK_INTERFACE` et `MOCK_AI`) et purge préventivement tout fichier `.env` ou `.env.exemple` résiduel dans `src/`.
- **Parsing booléen robuste (`ParseBoolFromString`)** : interprète les représentations usuelles (`true`, `1`, `yes`, `oui`, `false`, `0`, etc.).

15.3 Points clés

- **Isolation stricte** : Aucun fichier `.env` n'est créé ou lu dans `src/`, ni dans le projet externe analysé par l'outil.
- **Prise d'effet immédiate** : Les modifications saisies dans le TUI sont écrites sur disque et rechargées dynamiquement sans aucune recompilation du binaire Go.

15.4 Détails déplacés du code source

Précisions issues des commentaires retirés de `env.go` :

- `isAppRoot` vérifie si un répertoire est la racine Smart Commit (présence de `src/git_commit_release_no` exclusion des dossiers de base `src` / `cli`) ; `isTestDir` détecte les répertoires temporaires de test (préfixe `os.TempDir()` ou sous-chaîne `Test`, ex. `t.TempDir()`).
- `FindAppRoot` suit un ordre de recherche : hints valides d'abord, puis variable `SMART_COMMIT_APP_ROOT`, puis remontée depuis le code source Go (`runtime.Caller`), depuis l'exécutable Go, depuis le répertoire de travail, avec repli sur le premier hint non vide ou `"."` .
- `FindAppEnvPath` renvoie l'emplacement STRICTEMENT à la racine et ne cherche JAMAIS dans `src/` ni dans le dépôt cible ; `FindEnvPath` est un alias conservé pour compatibilité avec le code existant et les tests.
- `ParseBoolFromString` convertit `"true"`, `"1"`, `"yes"`, `"oui"` en booléen ; `ReadMockSettingFromEnv` / `ReadMockAISettingFromEnv` lisent respectivement `MOCK_INTERFACE`

(avec variante historique MOCKAGE_INTERFACE) et MOCK_AI, avec priorité aux variables d'environnement système directes puis lecture du .env applicatif.
- LoadConfigFromEnv purge par sécurité tout .env résiduel dans src/ ; WriteEnvFile réécrit le .env STRICTEMENT à la racine et supprime préventivement tout ancien src/.env pour ne garder que la racine du projet.

15.5 Commentaires de fonction (dupliqués du code source)

- isAppRoot() : Vérifie si un répertoire est la racine applicative.
- isTestDir() : Détecte si un chemin correspond à un répertoire temporaire de test.
- FindAppRoot() : Localise la racine applicative à partir des indices et de l'environnement.
- FindAppEnvPath() : Renvoie le chemin du fichier .env à la racine applicative.
- FindEnvPath() : Fournit un alias de compatibilité pour le chemin du .env.
- ParseBoolFromString() : Interprète une chaîne comme booléen.
- ReadMockSettingFromEnv() : Lit l'activation du mode simulation depuis l'environnement.
- ReadMockAISettingFromEnv() : Lit l'activation de la simulation IA depuis l'environnement.
- LoadConfigFromEnv() : Charge la configuration complète depuis le .env applicatif.
- WriteEnvFile() : Écrit la configuration dans le fichier .env applicatif.

---

16 Mock

17 Implémentation Simulée MockClient

Fichier source associé : src/cli/bridge/mock_client.go

17.1 Description

Ce module implémente l'interface BackendClient en fournissant des réponses synthétiques immédiates pour le mode démo hors-ligne et les tests automatisés du front-end.

17.2 Responsabilités

- Simuler un dépôt Git actif avec des modifications réalistes (détection de secrets, fichiers binaires, ajouts/suppressions).
- Générer des messages de commit simulés respectant la convention Conventional Commits.
- Produire des Release Notes formatées en Markdown sans appel réseau.
- Simuler la connectivité et la latence avec un serveur Ollama.
- Permettre la surcharge (CustomError, CustomDiff, SimulateDelay = false) pour les tests unitaires Go.

17.3 Points clés

- **Autonomie totale** : Permet de présenter l'application ou d'exécuter des tests d'interface sans nécessiter de serveur Ollama actif ni de backend Python installé.
- **Conformité aux règles de sécurité** : Démontre la détection des clés API sensibles et l'exclusion des fichiers binaires.

17.4 Détails déplacés du code source

Précisions issues des commentaires retirés de `mock_client.go` :

- `MockClient` fournit une implémentation de `BackendClient` pour les tests et le mode hors-ligne ; les champs `CustomRepoStatus`, `CustomDiff`, `CustomCommitProposal`, `CustomReleaseNotes`, `CustomPing` et `CustomError` permettent la surcharge pour les tests unitaires.
- `IsMock` retourne toujours `true` ; `SimulateDelay` (via `sleep`) simule les latences réseau.
- Données types : branche `interface-cli` avec 3 fichiers indexés et 1 non indexé ; `src/auth/token_vault.py` avec données sensibles masquées (`HasSecretsMasked`) ; `assets/banner.png` comme fichier binaire exclu.
- `GenerateCommit` renvoie une proposition Conventional Commits adaptée au `feedback` éventuel ; `ApplyCommit` applique le commit sur la branche courante ; `Push` envoie vers la branche distante ; `GetReleaseNotes` synthétise le Markdown entre deux tags ; `PingOllama` vérifie la joignabilité d'Ollama ; `SaveMarkdownFile` enregistre localement le Markdown exporté ; `GetConfig` / `SaveConfig` lisent et mettent à jour le `.env` applicatif.

17.5 Commentaires de fonction (dupliqués du code source)

- `NewMockClient()` : Instancie le client simulé avec la même racine pour dépôt et application.
- `NewMockClientWithAppRoot()` : Instancie le client simulé avec racines distinctes.
- `MockClient.sleep()` : Simule une latence réseau lorsque activée.
- `MockClient.IsMock()` : Indique que le client est toujours en mode simulation.
- `MockClient.GetRepoStatus()` : Retourne un statut de dépôt simulé.
- `MockClient.GetDiff()` : Retourne un diff simulé avec secrets et binaires.
- `MockClient.StageAll()` : Simule l'indexation de tous les fichiers.
- `MockClient.GenerateCommit()` : Génère une proposition de commit simulée adaptée au retour.
- `MockClient.ApplyCommit()` : Simule l'application du commit sur la branche courante.
- `MockClient.Push()` : Simule la poussée vers la branche distante.
- `MockClient.GetReleaseNotes()` : Génère des notes de version simulées entre deux tags.
- `MockClient.GetConfig()` : Charge la configuration simulée depuis l'environnement.
- `MockClient.SaveConfig()` : Persiste la configuration simulée dans le .env applicatif.
- `MockClient.PingOllama()` : Retourne un diagnostic Ollama simulé.
- `MockClient.SaveMarkdownFile()` : Sauvegarde le contenu Markdown dans le dépôt simulé.

---

18 Client Python

19 Implémentation Réelle PythonClient

Fichier source associé : src/cli/bridge/python_client.go

19.1 Description

Ce module implémente l'interface `BackendClient` en exécutant le backend Python (`git_commit_release_notes_generator.service.core`) sous la forme de sous-processus système via `os/exec`.

19.2 Responsabilités

- Résoudre dynamiquement l'interpréteur Python (`python3` ou environnement virtuel local `/home/maxence/monenv/bin/python`).
- Configurer les variables d'environnement du sous-processus : `PYTHONPATH` pointant sur `<appRoot>/src`, et transmission de `GITELLIGENCE_APP_ROOT` (avec maintien de `SMART_COMMIT_APP_ROOT` pour rétrocompatibilité).
- Exécuter les commandes métier (`--action status`, `diff`, `stage-all`, `generate-commit`, `apply-commit`, `push`, `release-notes`, `ping-ollama`).
- Isoler les flux : capture du JSON strict sur `stdout` et redirection des logs/traces sur `stderr`.
- Désérialiser les réponses JSON dans des structures Go fortement typées (`models`).

19.3 Points clés

- **Résilience** : En cas d'erreur du sous-processus, le message `stderr` est extrait et formaté pour un affichage propre sans panic.
- **Isolation du dépôt cible** : Le dépôt cible est transmis via `--repo`, tandis que le code et la configuration proviennent exclusivement de l'application.

19.4 Détails déplacés du code source

Précisions issues des commentaires retirés de `python_client.go` :

- `PythonClient` implémente `BackendClient` en communiquant avec le sous-processus Python et ne contient AUCUNE logique métier : tout le traitement est délégué à Python.
- `runPythonCommand` exécute la commande Python et récupère `stdout` ; `PYTHONPATH` pointe sur le dossier `src` de l'application, avec candidats de secours si `appSrc` n'est pas résolu ; la configuration est chargée exclusivement depuis l'application (`p.appRoot`, JAMAIS `p.repoRoot`).
- `IsMock` retourne toujours `false` ; chaque méthode (`GetRepoStatus`, `GetDiff`, `StageAll`, `GenerateCommit`, `ApplyCommit`, `Push`, `GetReleaseNotes`, `GetConfig`, `SaveConfig`, `PingOllama`, `SaveMarkdownFile`) interroge le backend Python pour l'action correspondante, `SaveConfig` notifiant Python après écriture du `.env` applicatif et `SaveMarkdownFile` écrivant localement le changelog exporté.

19.5 Commentaires de fonction (dupliqués du code source)

- `NewPythonClient()` : Instancie le client Python en résolvant la racine applicative.
- `NewPythonClientWithAppRoot()` : Instancie le client Python avec racines explicites et interpréteur résolu.
- `PythonClient.runPythonCommand()` : Exécute le module Python et capture sa sortie standard.
- `PythonClient.IsMock()` : Indique que le client réel n'est jamais en simulation.
- `PythonClient.GetRepoStatus()` : Interroge le backend Python pour le statut du dépôt.
- `PythonClient.GetDiff()` : Interroge le backend Python pour le diff.
- `PythonClient.StageAll()` : Demande au backend Python d'indexer tous les fichiers.
- `PythonClient.GenerateCommit()` : Demande au backend Python une proposition de commit.
- `PythonClient.ApplyCommit()` : Demande au backend Python d'appliquer le commit.
- `PythonClient.Push()` : Demande au backend Python de pousser vers le distant.
- `PythonClient.GetReleaseNotes()` : Demande au backend Python les notes de version.
- `PythonClient.GetConfig()` : Charge la configuration réelle depuis l'environnement.
- `PythonClient.SaveConfig()` : Persiste la configuration puis notifie le backend Python.
- `PythonClient.PingOllama()` : Interroge le backend Python sur l'état d'Ollama.

- `PythonClient.SaveMarkdownFile()` : Sauvegarde le contenu Markdown dans le dépôt cible.

---

20 Modèles

21 Modèles de Données Typées CLI

Fichier source associé : src/cli/models/types.go

21.1 Description

Ce module regroupe l'ensemble des structures de données Go représentant les échanges JSON entre le front-end Go et le back-end Python. Il assure un typage strict et évite les manipulations de chaînes de caractères brutes.

21.2 Structures principales

21.2.1 RepoStatus

Représente l'état contextuel du dépôt Git transmis par Python : - `IsGitRepo` (`bool`) : indique si le répertoire courant est un dépôt Git valide ; - `RepoPath` (`string`) : chemin absolu de la racine du dépôt ; - `Branch` (`string`) : nom de la branche courante ; - `StagedCount` (`int`) : nombre de fichiers indexés (`staged`) prêts pour le commit ; - `UnstagedCount` (`int`) : nombre de fichiers modifiés non indexés.

21.2.2 DiffFile

Détail d'un fichier extrait du diff Git par le backend : - `Path` (`string`) : chemin relatif du fichier modifié ; - `Status` (`string`) : type d'opération (`M` pour modifié, `A` pour ajouté, `D` pour supprimé) ; - `Added` / `Removed` (`int`) : volume de lignes ajoutées et supprimées ; - `Binary` (`bool`) : indicateur de fichier binaire (exclu de l'analyse LLM) ; - `HasSecretsMasked` (`bool`) : signal visuel indiquant si un secret/token a été assaini par sécurité.

21.2.3 CommitProposal

Proposition de commit générée par le modèle LLM au format Conventional Commits : - `Type` (`string`) : catégorie conventionnelle (`feat`, `fix`, `docs`, `refactor`, etc.) ; - `Scope` (`string`, optionnel) : sous-système concerné ; - `Subject` (`string`) : message descriptif concis ; - `Body` (`string`, optionnel) : explications détaillées de la modification ; - `RawFormatted` (`string`) : message assemblé prêt pour `git commit`.

21.2.4 ConfigResponse & PingResponse

- `ConfigResponse` : paramètres de configuration lus dans le `.env` applicatif (`OllamaBaseURL`, `OllamaModel`, `TimeoutS`, `Language`, `MockInterface`, `MockAI`).
- `PingResponse` : diagnostic réseau vers le serveur Ollama de l'IUT (latence en millisecondes et liste des modèles disponibles).

21.3 Rôle dans l'architecture

Contrat d'interface formel entre la sortie standard JSON du backend Python et les vues d'affichage en Go.

21.4 Détails déplacés du code source

Précisions issues des commentaires retirés de `types.go` et non redondantes avec les sections ci-dessus :

- `DiffResponse` contient la liste des fichiers en attente (`Files`) et les métadonnées associées (`Success`, `Error`, `StagedCount`).
- `ActionResult` représente le retour d'une action exécutée par Python (`commit`, `stage`, `push`, `save`), avec `Message`, `Sha` et `Error` optionnels.
- `ReleaseNotesResponse` contient le texte Markdown synthétisé par le LLM (`Markdown`), encadré par `FromTag` / `ToTag` et `CommitCount`.
- Le champ `Status` de `DiffFile` reprend les codes Git usuels (`M`, `A`, `D`, `R`, etc.).
- `RepoStatus` représente l'état brut du dépôt fourni par le backend Python ; `CommitProposal` représente la suggestion au format Conventional Commits ; `ConfigResponse` contient les paramètres lus ou écrits dans le `.env` par Python ; `PingResponse` contient le diagnostic de joignabilité du serveur Ollama de l'IUT.

---

22 Boîtes

23 Composants de Boîtes et Tableaux UI

Fichier source associé : src/cli/ui/box.go

23.1 Description

Ce module assemble les primitives de style pour construire des composants visuels d'interface riches : bannières contextuelles, cadres arrondis Unicode, tableaux de modifications Git et cartes de messages Conventional Commits.

23.2 Responsabilités

- `PrintBanner` : affiche la bannière d'en-tête dynamique du projet avec le statut Git fourni par Python (dépôt, branche, nombre de fichiers staged/non-staged) ;
- `PrintCard` : trace des panneaux encadrés rectilignes avec titre centré et couleur de bordure personnalisée ;
- `PrintDiffTable` : affiche le tableau récapitulatif des fichiers modifiés avec coloration syntaxique du statut (`[M]` jaune/cyan, `[A]` vert, `[D]` rouge), métriques de lignes +/-, et badge d'avertissement `[Secret Masqué]` ;
- `PrintCommitCard` : présente la proposition de commit mise en valeur (Type en vert gras, Scope en jaune, Sujet en blanc gras, Body séparé par un filet discret) ;
- `PrintErrorCard` : affiche un encadré d'erreur rouge harmonisé avec titre, message principal découpé automatiquement et liste de suggestions/actions recommandées ;
- `PrintSuccess`, `PrintWarning`, `PrintInfo` : affichent des alertes typées et colorées standardisées.

23.3 Points clés

- Rendu professionnel basé sur les caractères de traçage de boîte Unicode (┌, ┐, ┘, └, ─, │) ;
- Adaptation automatique de la largeur des cadres (`width`) ;
- Visibilité immédiate des mécanismes de sécurité (secrets masqués).

23.4 Rôle dans l'architecture

Bibliothèque de composants graphiques d'ordre supérieur consommée par les vues interactives (`views`).

23.5 Commentaires de fonction (dupliqués du code source)

- `PrintBanner()` : Affiche la bannière d'en-tête avec le statut du dépôt Git.
- `PrintCard()` : Affiche un panneau encadré avec titre et lignes de contenu.
- `PrintDiffTable()` : Affiche le tableau des fichiers modifiés du diff indexé.
- `PrintCommitCard()` : Affiche la proposition de commit conventionnel dans une carte.
- `PrintSuccess()` : Affiche un message de succès standardisé en vert.
- `PrintError()` : Affiche un message d'erreur standardisé en rouge.
- `PrintWarning()` : Affiche un message d'avertissement standardisé en jaune.
- `PrintInfo()` : Affiche un message d'information standardisé en cyan.

---

24 Spinner

25 Animation Spinner et Gestion Asynchrone

Fichier source associé : src/cli/ui/spinner.go

25.1 Description

Ce module gère un indicateur de chargement animé (spinner) non-bloquant basé sur une goroutine Go, apportant un retour visuel en temps réel pendant les opérations asynchrones (inférence LLM, récupération Git, ping réseau).

25.2 Responsabilités

- démarrer une goroutine indépendante affichant une séquence de caractères animés braille (⠋ ⠙ ⠹ ⠸ ⠼ ⠴ ⠦ ⠧ ⠇ ⠏) à une fréquence de 80ms ;
- afficher un chronomètre dynamique en temps réel mesurant le temps d'attente (ex: `(3.4s)`) ;
- masquer le curseur du terminal pendant l'animation pour éviter les clignotements parasites (`HideCursor`) ;
- mettre à jour dynamiquement le texte d'attente (`UpdateLabel`) ;
- stopper proprement l'animation à l'aide d'un channel de synchronisation (`close(stopChan)`) et restaurer le curseur (`ShowCursor`).

25.3 Points clés

- **Thread-safe** : protection des accès concurrents au libellé et à l'état actif via un verrou d'exclusion mutuelle (`sync.Mutex`) ;
- **Non-bloquant** : permet au thread principal d'attendre la réponse du backend Python sans figer l'interface ;
- **Nettoyage propre de la ligne** : réécriture en place via `\r\033[K` pour une transition sans saut de ligne indésirable.

25.4 Rôle dans l'architecture

Composant d'attente visuelle fournissant le feedback indispensable à l'utilisateur lors des requêtes d'inférence LLM vers le serveur distant d'Ollama.

25.5 Commentaires de fonction (dupliqués du code source)

- `NewSpinner()` : Crée un nouveau spinner avec le libellé indiqué.
- `Spinner.Start()` : Démarre l'animation du spinner dans une goroutine.
- `Spinner.UpdateLabel()` : Met à jour le libellé affiché par le spinner.
- `Spinner.Stop()` : Arrête le spinner et affiche éventuellement un message final.

---

26 Styles

27 Moteur de Styles et Séquences ANSI

Fichier source associé : src/cli/ui/styles.go

27.1 Description

Ce module fournit les constantes d'échappement ANSI et les fonctions utilitaires pour le formatage visuel du terminal : couleurs vives, couleurs TrueColor 24-bit, contrôle du curseur et calcul précis des largeurs visuelles.

27.2 Responsabilités

- définir les constantes ANSI fondamentales (`Reset`, `Bold`, `Dim`, `Italic`, `Underline`, `ClearScreen`, `HideCursor`, `ShowCursor`) ;
- fournir une palette de couleurs standard (`Cyan`, `Yellow`, `Green`, `Red`, `Blue`, `Magenta`, `White`, `Gray`) ;
- supporter le rendu TrueColor RGB (`Rgb`, `NeonGradient`) ;
- calculer la largeur visible réelle d'une chaîne contenant des codes d'échappement ANSI (`VisualLen`) via la suppression d'expressions régulières (`StripANSI`) ;
- assurer le remplissage et l'alignement de chaînes (`PadRight`, `PadLeft`, `PadCenter`) sans décaler les bordures des cadres ;
- réinitialiser complètement l'écran et le tampon d'historique de défilement (`ClearTerminal`).

27.3 Points clés

- **Alignement au pixel/caractère près** : l'utilisation naïve de `len(string)` en Go fausse le calcul dès qu'une couleur ANSI ou un caractère Unicode accentué est présent. `VisualLen` compte le nombre exact de runes visibles affichées par le terminal ;
- **Nettoyage absolu** : envoi de la séquence `\033[H\033[2J\033[3J` pour un écran totalement propre à chaque changement de vue.

27.4 Rôle dans l'architecture

Socle de présentation graphique de bas niveau utilisé par tous les composants visuels et toutes les vues du CLI.

27.5 Commentaires de fonction (dupliqués du code source)

- `StripANSI()` : Supprime les séquences d'échappement ANSI d'une chaîne.
- `VisualLen()` : Calcule la longueur visible d'une chaîne sans les codes ANSI.
- `PadRight()` : Aligne une chaîne à gauche en complétant à droite.
- `PadLeft()` : Aligne une chaîne à droite en complétant à gauche.
- `PadCenter()` : Centre une chaîne en complétant des deux côtés.
- `ClearTerminal()` : Efface complètement le terminal et son historique de défilement.
- `Rgb()` : Colore un texte avec une couleur TrueColor RGB.
- `NeonGradient()` : Applique un dégradé néon animé à un texte.

---

28 Menu

29 Vue Menu Principal

Fichier source associé : src/cli/views/menu_view.go

29.1 Description

Ce module gère l'affichage de la page d'accueil et la capture du premier niveau d'interaction de l'utilisateur.

29.2 Responsabilités

- effacer l'écran et afficher la bannière contextuelle du projet ;
- présenter les 4 choix d'actions numérotés :
  - `[1]` Générer un message de commit (IA Conventional Commits) ;
  - `[2]` Générer les Release Notes (Synthèse entre tags) ;
  - `[3]` Configuration (Modèle, URL Ollama, Timeout) ;
  - `[4]` Quitter ;
- signaler distinctement si le mode simulation de l'interface (`Mock Interface actif`) et/ou la simulation de l'IA (`Mock IA actif`) sont actifs (les deux s'affichent si les deux options sont activées) ;
- capturer la saisie utilisateur de manière robuste via `ReadChoice`.

29.3 Points clés

- **Simplicité d'utilisation** : navigation rapide par touche directe (1, 2, 3, 4 ou q) ;
- **Contexte temps réel** : actualisation de la branche et du nombre de fichiers modifiés à chaque retour sur le menu ;
- **Saisie stable** : utilisation de `ReadChoice` avec prompt mono-ligne empêchant tout défilement en cascade lors d'appuis répétés sur Entrée à vide.

29.4 Rôle dans l'architecture

Écran racine du CLI, point de départ de toutes les fonctionnalités applicatives.

29.5 Commentaires de fonction (dupliqués du code source)

- `ShowMainMenu()` : Affiche le menu principal et retourne le choix utilisateur.

---

30 Commit

31 Vue Génération de Commit

Fichier source associé : src/cli/views/commit_view.go

31.1 Description

Ce module orchestre le workflow interactif de génération, de validation et d'application des messages de commit. Il structure le dialogue entre l'utilisateur et l'IA à travers une boucle de décision complète.

31.2 Workflow en 4 étapes

1. **Analyse du diff** : demande au backend Python la liste des fichiers modifiés et les affiche. Si aucun fichier n'est indexé, propose une indexation automatique (`git add -A`) via Python.
2. **Génération par le LLM** : déclenche le spinner d'attente pendant que Python interroge le modèle distant.
3. **Restitution structurée** : affiche la carte du message proposé au format Conventional Commits.
4. **Boucle de décision utilisateur** :
   - `[V]` **Valider** : applique le commit via Git et propose d'enchaîner sur un `git push` (option masquée et bloquée si le message est en erreur, jusqu'à modification manuelle [M] ou régénération [R]) ;
   - `[R]` **Régénérer avec consigne** : demande à l'utilisateur un texte de feedback (ex: "sois plus concis", "c'est un fix") et relance l'inférence LLM avec ce contexte ;
   - `[M]` **Modifier manuellement** : permet d'ajuster le type, le scope, le sujet ou le corps directement dans le terminal avant validation ;
   - `[A]` **Annuler** : abandonne l'opération sans impacter le dépôt Git.

31.3 Points clés

- **Respect absolu du rôle front-end** : aucune commande Git directe n'est exécutée depuis ce fichier ; toutes les requêtes (`stage`, `commit`, `push`, `llm`) sont transmises au `BackendClient` ;
- **Contrôle utilisateur total** : l'utilisateur reste maître absolu de la validation finale du commit ;
- **Fonction `ReadChoice`** : utilitaire de saisie partagé absorbant les appuis sur Entrée à vide in-situ via des séquences ANSI (`\033[1A\r` + `\033[K`) avec préservation de l'espace après l'emoji, sans générer de sauts de ligne intempestifs ;
- **Architecture Event Loop (Clean Code)** : découplage strict entre la génération initiale et la boucle d'interaction utilisateur, avec appel direct du LLM dans le choix `[R]` et édition manuelle dédiée (`promptManualEdit`).

31.4 Rôle dans l'architecture

Vue maîtresse du sujet 8, réalisant l'automatisation de la rédaction des messages de commit assistée par intelligence artificielle.

31.5 Détails déplacés du code source

Anciens commentaires internes de `ShowCommitFlow` (supprimés du `.go`, conservés ici) : le flux est séquencé en 5 étapes numérotées dans le code — (1) récupération du diff via le backend Python, (2) gestion du cas « aucun fichier staged » avec proposition d'indexation automatique puis re-téléchargement du diff actualisé, (3) affichage du tableau des fichiers modifiés, (4) boucle de génération et de raffinement avec le LLM, (5) menu de décision utilisateur (V/R/M/A), avec proposition de push après un commit validé.

31.6 Commentaires de fonction (dupliqués du code source)

- `ShowCommitFlow()` : Orchestre le flux interactif de génération et validation du commit.
- `formatCommit()` : Formate un message de commit conventionnel à partir de ses parties.
- `waitForEnter()` : Attend l'appui sur Entrée avant de continuer.

---

32 Config

33 Vue Configuration

Fichier source associé : src/cli/views/config_view.go

33.1 Description

Ce module permet de visualiser et de modifier les paramètres opérationnels de l'application (URL Ollama, modèle LLM, timeout) et de diagnostiquer la connectivité réseau avec le serveur de l'IUT.

33.2 Responsabilités

- afficher les paramètres actuellement appliqués issus du fichier d'environnement `.env` situé à la racine du projet (URL, Modèle, Timeout, ainsi que les statuts distincts `Mock`

`Interface` et `Mock IA` exclusivement lorsque le mode `--dev` est actif) ;
- **Action [T] (Test Ping)** : exécuter un diagnostic réseau vers l'API Ollama (ou simulation instantanée si Mock IA actif), mesurer la latence en millisecondes et lister l'ensemble des modèles d'IA installés sur le serveur ;
- **Action [M] (Modification)** : permettre la saisie de nouvelles valeurs et persister les modifications dans le fichier `.env` à la racine de l'application sans aucune recompilation du binaire (la présence et la modification interactive des options de mock `Mock Interface` et `Mock IA` sont réservées au mode `--dev`) ;
- **Action [R]** : retour au menu principal.

33.3 Points clés

- **Conformité stricte aux exigences d'évaluation** : respect de la règle d'or "tout doit être dans un `.env`, aucune recompilation requise lors d'un changement de modèle ou d'adresse" ;
- **Diagnostic intégré** : permet de valider instantanément si le serveur de l'IUT (`10.22.28.190:11434`) répond avant de lancer des opérations de génération ;
- **Saisie sécurisée** : utilisation de `ReadChoice` pour éviter toute sortie accidentelle lors d'un appui sur Entrée à vide (retour exigeant explicitement la touche `[R]`).

33.4 Rôle dans l'architecture

Interface d'administration et de diagnostic du front-end, garantissant la configurabilité à chaud du système.

33.5 Commentaires de fonction (dupliqués du code source)

- `ShowConfigFlow()` : Affiche et gère le flux interactif de configuration.

---

34 Release notes

35 Vue Release Notes

Fichier source associé : src/cli/views/release_notes_view.go

35.1 Description

Ce module gère l'écran interactif dédié à la synthèse de version et à la production automatique de changelogs au format Markdown entre deux bornes de révision Git (tags).

35.2 Responsabilités

- inviter l'utilisateur à saisir le tag de départ (par défaut `v0.1.0`) et le tag d'arrivée (par défaut `HEAD`) ;
- déclencher la synthèse des commits auprès du backend Python avec retour visuel animé ;
- afficher le document Markdown catégorisé dans un panneau encadré ;
- proposer l'enregistrement direct du résultat dans un fichier `RELEASE_NOTES.md` à la racine du projet (option masquée et bloquée en cas d'erreur de génération).

35.3 Points clés

- **Catégorisation sémantique** : exploitation de la classification produite par le LLM (Nouveautés, Corrections de bogues, Documentation & Maintenance) ;
- **Export en 1 touche** : écriture immédiate du fichier Markdown sans quitter l'interface ;
- **Boucle de décision robuste** : utilisation de `ReadChoice` pour absorber les appuis vides sur Entrée sans quitter l'écran et verrouillage contextuel de la sauvegarde `[S]`.

35.4 Rôle dans l'architecture

Vue secondaire du sujet 8, concrétisant la seconde fonctionnalité majeure de la SAÉ : la génération automatisée de notes de version.

35.5 Détails déplacés du code source

Anciens commentaires internes de `ShowReleaseNotesFlow` (supprimés du `.go`, conservés ici) : la fonction se structure en trois phases marquées dans le code — saisie des bornes de révision (`fromTag`/`toTag`), génération puis affichage du Markdown dans un panneau encadré, et choix utilisateur final (S sauvegarder / R retour menu).

35.6 Commentaires de fonction (dupliqués du code source)

- `ShowReleaseNotesFlow()` : Orchestre le flux interactif de génération des release notes.

---

36 init

37 Module git_commit_release_notes_generator

Fichier source associé : src/git_commit_release_notes_generator/**init**.py

37.1 Rôle

Ce package regroupe le cœur de la génération des messages de commit et des notes de release. Il centralise les données de configuration, les structures de données partagées et les clients externes qui interagissent avec Ollama.

37.2 Sous-modules

- config.md : paramètres de configuration de l'application.
- models.md : modèles de données partagés.
- ollama_client : couche de communication avec Ollama.
- service : logique métier et traitements applicatifs.

37.3 Objectif fonctionnel

Le package sert de point d'entrée logique pour :

- analyser un diff Git ;
- filtrer les données sensibles ;
- transmettre un contexte structuré au modèle LLM ;

- générer un message de commit conforme à Conventional Commits ;
- produire éventuellement des éléments exploitables pour des notes de version.

---

38 config

39 Configuration

Fichier source associé : src/git_commit_release_notes_generator/config.py

39.1 Description

Ce module contient les paramètres globaux utilisés par le client Ollama et le reste de l'application.

39.2 Variables principales

- `OLLAMA_BASE_URL` : URL du serveur Ollama (ex: `http://10.22.28.190:11434` ou local).
- `OLLAMA_MODEL` : modèle utilisé pour la génération des messages de commit (ex: `gemma4:26b`, `gemma4:12b`).
- `OLLAMA_TIMEOUT_S` : délai de requête avant timeout en secondes.
- `MOCK_INTERFACE` : activation du mode simulation complète de l'interface en Go (`true` ou `false`).
- `MOCK_AI` : activation de la simulation des réponses de l'IA Ollama hors-ligne (`true` ou `false`).

39.3 Emplacement et chargement du fichier `.env`

- **Emplacement strict** : Le fichier `.env` réside **exclusivement à la racine du dépôt du projet** (`SAE_sujet_8_application_intelligente/.env`), ou à l'emplacement indiqué par la variable d'environnement système `GITELLIGENCE_APP_ROOT` (avec repli sur `SMART_COMMIT_APP_ROOT`).
- **Aucune pollution** : Le module ne lit ni n'écrit jamais de `.env` dans `src/` ou dans le répertoire de travail courant `Path.cwd()`, évitant toute confusion ou parasitage d'un dépôt Git tiers analysé par l'outil.
- **Surcharge** : Les variables définies directement dans l'environnement système ont priorité sur le contenu du fichier `.env`.

39.4 Points de vigilance

- La configuration est lue dynamiquement sans nécessiter de recompilation ni de modification de code.
- Les valeurs par défaut restent compatibles avec l'environnement de l'IUT et la CI.
- Le fichier modèle `.env.exemple` est versionné à la racine pour guider le déploiement initial.

39.5 Exemple d'usage

Le client LLM et le service `core.py` importent cette configuration pour construire les requêtes HTTP vers Ollama et initialiser les paramètres d'exécution.

39.6 Détails déplacés du code source

- `parse_bool(val)` : accepte un `bool` (retourné tel quel), ou une chaîne insensible à la casse et aux espaces. Valeurs vraies : `"true"`, `"1"`, `"yes"`, `"oui"`. Toute valeur absente, vide ou `None` vaut `False`.
- `load_env()` : construit une liste de chemins candidats (`.env` sous `SMART_COMMIT_APP_ROOT` si défini, puis `.env` à la racine du dépôt déduite de `Path(__file__).resolve().parent.parent`). Chaque candidat est dédupliqué via `resolve()`, seul le premier fichier existant est lu ; chaque ligne est nettoyée (`strip()`, guillemets retirés) et les lignes vides, commençant par `#` ou sans `=` sont ignorées. Les variables système `OLLAMA_BASE_URL`, `OLLAMA_MODEL`, `OLLAMA_TIMEOUT_S`, `OLLAMA_NUM_CTX`, `APP_LANGUAGE`, `MOCK_INTERFACE`, `MOCK_AI` surchargent ensuite le fichier si elles sont non vides.
- `OLLAMA_NUM_CTX` vaut `"32768"` par défaut : sans ce paramètre explicite, Ollama utilise sa fenêtre de contexte réduite par défaut (souvent 2048-4096 tokens selon le `Modelfile`) même si le modèle supporte bien plus, et tronque silencieusement le début du contexte sur les diffs volumineux.

39.7 Commentaires de fonction (dupliqués du code source)

- `parse_bool()` : Convertit une valeur texte ou booléenne en booléen.
- `load_env()` : Charge les variables depuis le fichier .env et l'environnement système.

---

40 models

41 Modèles de données

Fichier source associé : src/git_commit_release_notes_generator/models.py

41.1 Description

Ce module définit les structures de données transverses utilisées entre les composants du projet.

41.2 Structures principales

41.2.1 DiffFile

Représente un fichier modifié dans un diff Git.

Champs importants :

- path : chemin du fichier ;
- status : état du fichier (A, M, D, R, etc.) ;
- added / removed : nombre de lignes ajoutées et supprimées ;
- is_binary : indique si le fichier est binaire ;
- patch : contenu brut du diff.

41.2.2 CommitMessage

Représente le message final de commit produit par le modèle.

Champs importants :

- type : type Conventional Commits (feat, fix, docs, chore, etc.) ;
- scope : portée optionnelle ;
- subject : sujet principal ;
- body : description détaillée optionnelle.

41.3 Rôle dans le flux

Les objets de ce module servent de contrat entre :

- le parsing Git ;
- la génération de texte via Ollama ;
- l'assemblage final du message de commit.

Ils évitent les chaînes non structurées et rendent le code plus robuste et testable.

41.4 Détails déplacés du code source

- Contrat d'équipe : ne pas modifier ces structures sans en discuter avec l'équipe. `DiffFile` est produit par le module de parsing Git (module B) et `CommitMessage` est consommé par la couche d'affichage Go.
- `DiffFile.status` : codes de statut Git (`A`, `M`, `D`, `R`, etc.). Le constructeur accepte `is_binary` ou l'alias `binary` (mot-clé uniquement) ; si les deux sont fournis avec des valeurs divergentes, il lève `TypeError`, sinon `is_binary` prime, défaut `False`.
- Propriété `DiffFile.binary` : alias rétrocompatible avec l'ancien contrat historique, renvoie `is_binary`.
- `CommitMessage.type` : types Conventional Commits (`feat`, `fix`, `chore`, `docs`, `refactor`, `test`, `perf`, `build`, `ci`).

41.5 Commentaires de fonction (dupliqués du code source)

- `DiffFile.__init__()` : Initialise un fichier de diff en réconciliant les indicateurs binaires.
- `DiffFile.binary()` : Expose l'indicateur binaire via l'alias historique.
- `DiffFile.to_dict()` : Sérialise le fichier de diff en dictionnaire.

---

42 init

43 Package ollama_client

Fichier source associé : src/git_commit_release_notes_generator/ollama_client/**init**.py

43.1 Rôle

Ce package encapsule la communication avec Ollama pour la génération de messages de commit.

43.2 Sous-modules

- base.md : primitives de base et point d'entrée technique.
- llm.md : logique de génération de message et validation du résultat.
- embedding.md : gestion des embeddings, si cette fonctionnalité est ajoutée plus tard.

- exceptions.md : exceptions métier du client.
- prompts : prompt système utilisé pour imposer la structure du message.

43.3 Objectif

Le package isole les dépendances externes LLM du reste du projet afin de garder la logique métier testable et plus simple à maintenir.

---

44 base

45 Base Ollama client

Fichier source associé : src/git_commit_release_notes_generator/ollama_client/base.py

45.1 Description

Ce module sert de base commune au client Ollama. Il définit les contrats et les éléments de fondation pour les intégrations avec le modèle.

45.2 Rôle attendu

- centraliser les interfaces de communication ;
- préparer les appels HTTP ;
- exposer une structure cohérente pour les autres sous-modules.

45.3 Utilisation

Les modules spécialisés reposent sur cette base pour éviter de dupliquer la logique de transport et de gestion des erreurs.

---

46 embedding

47 Embedding Ollama

Fichier source associé : src/git_commit_release_notes_generator/ollama_client/embedding.py

47.1 Description

Ce module est dédié aux embeddings, c'est-à-dire à la transformation de texte en vecteurs numériques pour les traitements de similarité ou d'indexation.

47.2 Statut actuel

Le fichier est prévu pour évoluer avec les besoins du projet et peut servir à des cas d'usage futurs, notamment la recherche ou la comparaison de diff/commit.

47.3 Rôle fonctionnel attendu

- convertir des chaînes textuelles en représentations vectorielles ;
- collaborer avec d'autres modules de recherche ou de classification ;
- rester isolé de la logique métier principale.

---

48 exceptions

49 Exceptions Ollama

Fichier source associé : src/git_commit_release_notes_generator/ollama_client/exceptions.py

49.1 Description

Ce module définit les erreurs spécifiques à la couche LLM et aux échanges avec Ollama.

49.2 Exceptions prévues

- OllamaCallError : échec de l'appel au serveur.
- CommitMessageValidationError : validation du JSON de réponse impossible ou incohérente.

49.3 Pourquoi les centraliser ?

Cela permet de distinguer clairement :

- les erreurs réseau ;
- les erreurs de format de réponse ;
- les erreurs métier liées au contenu généré.

Cette séparation simplifie le diagnostic et le test des cas limites.

---

50 llm

51 Client LLM

Fichier source associé : src/git_commit_release_notes_generator/ollama_client/llm.py

51.1 Description

Ce module gère la génération du message de commit via Ollama. Il construit le résumé du diff, envoie une requête structurée à l'API de chat, valide la réponse JSON et la transforme en objet métier.

51.2 Flux de travail

1. Rejeter immédiatement une liste de fichiers vide (`ValueError`), sans appel réseau.
2. Lire les fichiers différences et les convertir en résumé exploitable.
3. Construire le prompt système et le message utilisateur.
4. Appeler Ollama via une requête JSON structurée, avec une fenêtre de contexte (`num_ctx`) fixée explicitement.
5. Vérifier que la réponse est bien un JSON conforme.
6. En cas d'erreur, relancer une seconde fois avec le message invalidé en contexte.
7. Convertir la réponse validée en objet CommitMessage.

51.3 Points clés

- format : le modèle répond avec un schéma JSON strict ;
- validation : le code refuse les messages trop longs, absents ou non conformes ;
- robustesse : un retry est tenté pour corriger une réponse invalide ;
- fenêtre de contexte : `options.num_ctx` est fixé explicitement dans la requête (`OLLAMA_NUM_CTX`, config.py) car Ollama utilise sinon une fenêtre par défaut bien plus petite que celle annoncée pour le modèle, ce qui tronquait silencieusement les diffs volumineux ;
- disponibilité : `check_ollama_reachable()` permet de vérifier rapidement (quelques secondes) si le serveur répond, avant de lancer une génération qui pourrait sinon attendre le timeout complet ;
- simulation hors-ligne : si `config.MOCK_AI` est activé, `generate_commit_message` et `check_ollama_reachable` délèguent immédiatement au module `mock_llm` sans aucune requête réseau ;
- override déterministe du type : si le `feedback` utilisateur contient une formulation d'intention explicite ("c'est un fix", "je veux un feat", "type: chore") suivie d'un seul type Conventional Commits connu, `_extract_forced_type()` le détecte et `_apply_forced_type()` l'impose sur le `CommitMessage` final, quel que soit ce que le modèle a réellement répondu pour ce champ. Le modèle garde la main sur `subject`/`body`/`scope`. Si aucun type n'est cité, ou si plusieurs types contradictoires le sont, aucun override n'est appliqué. Une formulation d'intention est exigée (pas la simple présence du mot) car certains types (`test`) sont aussi des mots français courants.

51.4 Fonctions exposées

- `generate_commit_message(diff_files, feedback="")` : fonction principale, décrite ci-dessus. Le `feedback` sert à la fois de consigne de révision injectée dans le prompt et, si un type y est explicitement cité, de source d'override déterministe du type final.
- `check_ollama_reachable(timeout=3.0, fetch_models=False)` (alias `ping_ollama`) : fonction unique et centralisée de diagnostic Ollama. Vérifie la joignabilité (`/api/version`), mesure la latence réseau en millisecondes et extrait optionnellement les modèles installés (`/api/tags` si `fetch_models=True`). Retourne un objet `PingResult(reachable, error, latency_ms, installed_models)` déballable directement en tuple `(reachable, error)` ou sérialisable via `.to_dict()`. Utilisée de manière transverse par `action_generate_commit`, `action_release_notes` et `action_ping_ollama` (écran de configuration Go).

51.5 Dépendances

- config.md
- models.md

- mock_llm.md
- exceptions.md
- commit_message_system.md

51.6 Détails déplacés du code source

- `PingResult` : outre l'accès par attributs (`.reachable`, `.latency_ms`, `.error`, `.installed_models`), il supporte le déballage en tuple (`reachable, error = check_ollama_reachable()`) via `__iter__`/`__getitem__`, et la comparaison directe à un tuple de 2 éléments via `__eq__`. `to_dict()` renvoie `success`, `reachable`, `latency_ms`, `installed_models`, et n'ajoute la clé `error` que si elle est non vide.
- `check_ollama_reachable()` : ne lève jamais d'exception ; mesure la latence via `/api/version`, et n'interroge `/api/tags` que si `fetch_models=True` (échec de récupération des modèles seulement tracé en debug).
- `generate_commit_message()` : un seul retry est tenté ; la réponse invalide est réinjectée comme message `assistant` sérialisé en JSON, suivie d'un message `user` signalant l'erreur constatée et exigeant un JSON conforme. Si le second essai échoue, lève `CommitMessageValidationError` plutôt que de renvoyer un message bancal.
- `_build_diff_summary()` : assemble un résumé textuel `Fichier: {path} (statut={status}, +{added}/-{removed})` suivi du `patch`, fichiers joints par une double newline.
- `_validate(raw)` : retourne `None` si valide, sinon un message explicite (`type` hors liste autorisée, `subject` manquant ou vide, `subject` de plus de 72 caractères, `subject` se terminant par un point).
- `_to_commit_message(raw)` : mappe `type`, `scope`, `subject`, `body` du dictionnaire validé vers `CommitMessage`.
- `_extract_forced_type(feedback)` : détecte si le feedback impose explicitement un type. Un préfixe d'intention explicite est requis avant le mot-clé de type (`c'est`, `je veux`, `mets en`, `type:`, ...) sinon un mot comme `test` matcherait n'importe quelle phrase française normale sans rapport avec une consigne. Si un seul type connu est trouvé, il est appliqué de façon déterministe ; si aucun ou plusieurs types contradictoires sont trouvés, retourne `None` et le modèle garde la main.
- `_apply_forced_type(msg, forced_type)` : écrase `msg.type` par le type imposé (avec trace `info` du type proposé par le modèle) ; appliqué aux trois sorties de `generate_commit_message` (mock, premier essai, retry).

51.7 Commentaires de fonction (dupliqués du code source)

- `_extract_forced_type()` : Détecte le type imposé explicitement par le retour utilisateur.
- `_apply_forced_type()` : Applique le type imposé au message généré si nécessaire.
- `PingResult.__iter__()` : Permet le déballage du résultat en tuple.
- `PingResult.__getitem__()` : Permet l'accès au résultat par index.
- `PingResult.__eq__()` : Compare le résultat à un tuple ou un objet.
- `PingResult.to_dict()` : Sérialise le résultat du diagnostic en dictionnaire.
- `check_ollama_reachable()` : Vérifie la joignabilité du serveur Ollama et mesure la latence.
- `generate_commit_message()` : Génère un message de commit à partir des diffs stagés.
- `_build_diff_summary()` : Assemble le résumé textuel des diffs pour le prompt.
- `_call_chat()` : Envoie les messages au chat Ollama et renvoie le JSON.
- `_validate()` : Valide la réponse brute et signale l'erreur éventuelle.
- `_to_commit_message()` : Convertit la réponse validée en objet CommitMessage.

---

52 mock_llm

53 Module de Simulation de l'IA (Mock LLM)

Fichier source associé : `src/git_commit_release_notes_generator/ollama_client/mock_llm.py`

53.1 Description

Ce module fournit une implémentation simulée (Mock) des fonctionnalités d'intelligence artificielle d'Ollama. Il permet de travailler sur l'application en mode hors-ligne sans serveur Ollama actif, tout en conservant l'analyse Git réelle, l'indexation et la persistance des fichiers.

53.2 Responsabilités

- Générer un message de commit déterministe et statique respectant le format Conventional Commits (`feat(mock): commit mocker pour test hors ligne`) ;
- Supporter la prise en compte de consignes utilisateur (`feedback`) injectées dans le corps du message pour les tests de révision interactive ;
- Simuler la charge utile JSON d'un appel à l'API de chat Ollama (`mock_call_chat`) ;
- Générer une réponse fixe et prévisible pour les Release Notes (`mock_release_notes`) ;
- Simuler le ping de connectivité et la liste des modèles installés (`mock_ping`) avec une latence factice de 10 ms.

53.3 Fonctions exposées

Fonction	Arguments	Retour	Description
`mock_generate_commit_message`	`diff_files`, `feedback=""`	`CommitMessage`	Renvoie le commit statique mocké. Lève `ValueError` si la liste de diff est vide.
`mock_call_chat`	`messages`	`dict`	Simule la réponse brute de `/api/chat`.
`mock_release_notes`	`commits`, `from_tag`, `to_tag`	`str`	Renvoie une note de version Markdown statique déterministe.
`mock_ping`	—	`dict`	Renvoie un diagnostic de connectivité simulé réussi avec latence de 10 ms.

53.4 Points clés

- **Simplicité et prévisibilité** : Les réponses sont volontairement statiques et prévisibles afin de faciliter les tests unitaires et le développement hors-ligne.
- **Délégation automatique** : `llm.py` et `core.py` basculent automatiquement vers ce module dès que la variable `MOCK_AI` est activée dans la configuration.

53.5 Commentaires de fonction (dupliqués du code source)

- `mock_generate_commit_message()` : Génère un message de commit simulé pour le mode hors-ligne.
- `mock_call_chat()` : Simule une réponse brute de l'API de chat.
- `mock_release_notes()` : Génère des release notes simulées entre deux révisions.
- `mock_ping()` : Simule un diagnostic de connectivité Ollama réussi.

---

54 Prompt système

55 Prompt système de génération de commit

Fichier source associé : src/git_commit_release_notes_generator/ollama_client/prompts/commit_message_sys

55.1 Description

Ce fichier contient le prompt système qui impose à l'IA de renvoyer un message de commit structuré et cohérent avec la convention Conventional Commits.

55.2 Objectif

Le prompt sert à guider le modèle vers une sortie stable avec :

- un type de commit valide ;
- un sujet concise ;
- une structure de message compréhensible ;
- un format exploitable par le code sans ambiguïté.

55.3 Rôle dans le flux

Il est utilisé par le module llm.md comme contexte système avant l'envoi du diff Git au modèle.

---

56 init

57 Service du générateur

Fichier source associé : src/git_commit_release_notes_generator/service/**init**.py

57.1 Rôle

Ce package rassemble la logique métier pure de l'application. Il sert à orchestrer les traitements et à exposer les fonctions qui manipulent les diff et les données de génération.

57.2 Sous-modules

- core.md : noyau métier du système.
- utils.md : utilitaires de traitement et aide fonctionnelle.

57.3 Objectif

Cette couche permet de garder le code de génération séparé des détails de communication réseau ou de parsing Git.

57.4 Détails déplacés du code source

- Le paquet réexporte `DiffFile` (depuis `models`), ainsi que `GitWrapper` et `GitWrapperError` (depuis `service.git_wrapper`) pour offrir un point d'accès unique à la couche service.

---

58 core

59 Core du service (Point d'entrée du pont Go ↔ Python)

Fichier source associé : src/git_commit_release_notes_generator/service/core.py

59.1 Description

Ce module constitue le point d'entrée exécutable sous forme de sous-processus par le front-end Go (`src/cli/bridge/python_client.go`). Il orchestre l'ensemble des fonctionnalités du back-end Python : - interaction et extraction des statuts et diffs Git via `GitWrapper` ; - filtrage des fichiers et masquage des secrets ; - interrogation du LLM via `generate_commit_message` ; - synthèse des release notes au format Markdown ; - diagnostic et joignabilité du serveur Ollama.

59.2 Contrat d'interface (Règle d'or Go / Python)

1. **Sortie JSON stricte sur `stdout`** : seul un objet JSON unique est imprimé sur la sortie standard.
2. **Logs et diagnostics sur `stderr`** : toutes les traces, messages d'information et erreurs passent exclusivement par le module `logging` sur `sys.stderr`.
3. **Résilience sans crash** : toutes les exceptions sont interceptées pour renvoyer un JSON d'erreur propre `{"success": false, "error": "..."}`.

59.3 Les 9 actions gérées (`--action <NOM>`)

`--action`	Arguments	Format de sortie JSON	Rôle et source des données
`status`	`--repo`	`{"is_git_repo", "repo_path", "branch", "staged_count", "unstaged_count"}`	Détermine l'état du dépôt Git via `GitWrapper`
`diff`	`--repo`	`{"success", "files": [...], "staged_count"}`	Analyse et assainit les fichiers indexés (`GitWrapper._parse_diff`)

--action	Arguments	Format de sortie JSON	Rôle et source des données
stage-all	--repo	{"success", "message"}	Indexe l'ensemble des fichiers modifiés (git add -A)
generate-commit	--repo, --feedback	{"success", "type", "scope", "subject", "body", "raw_formatted"}	Inférence du commit Conventional Commits avec support du feedback
apply-commit	--repo, --message	{"success", "sha", "message"}	Applique le commit sur la branche courante via GitWrapper.commit()
push	--repo	{"success", "message"}	Pousse vers le dépôt distant configuré via git push
release-notes	--repo, --from, --to	{"success", "from_tag", "to_tag", "markdown", "commit_count"}	Synthèse Markdown des commits entre deux révisions (LLM + repli déterministe)
save-config	--base-url, --model, --timeout, --language	{"success", "message"}	Notification de sauvegarde de configuration
ping-ollama	—	{"success", "reachable", "latency_ms", "installed_models"}	Vérification HTTP de joignabilité et des modèles disponibles sur Ollama

59.4 Fonctions clés

- action_status(repo_path) : vérifie la validité du dépôt, gère les branches orphelines (detached HEAD) et dénombre les modifications.
- action_diff(repo_path) : expose chaque fichier avec son patch, ses compteurs added/removed, le drapeau binary et la détection has_secrets_masked.
- action_generate_commit(repo_path, feedback) : extrait le diff staged textuel, teste la joignabilité d'Ollama via check_ollama_reachable() pour échouer rapidement en cas de problème réseau, et le transmet au module LLM avec la consigne optionnelle utilisateur (ou renvoie le commit statique si MOCK_AI=True).
- action_release_notes(repo_path, from_tag, to_tag) : extrait l'historique, teste la joignabilité d'Ollama via check_ollama_reachable() (échoue immédiatement avec success=False si injoignable, à l'identique de action_generate_commit) et génère une synthèse catégorisée (Features, Fixes, Refactoring, Docs) via le LLM avec repli déterministe si l'appel chat échoue (ou renvoie la synthèse statique si MOCK_AI=True).
- action_ping_ollama() : teste la joignabilité, mesure la latence et liste les modèles installés en déléguant à la fonction unifiée check_ollama_reachable(fetch_models=True).

59.5 Détails déplacés du code source

- Exécution en sous-processus : python -m git_commit_release_notes_generator.service.core --action <NOM> [args], appelé par src/cli/bridge/python_client.go. Le proces-

sus sort toujours en code 0, même pour une action inconnue, car le client Go ignore entièrement `stdout` si le code de sortie est non nul.

- `action_status` : la branche courante gère le `detached HEAD` et le commit initial (repli sur les 7 premiers caractères du SHA, sinon `"HEAD"`). `staged_count` compte les lignes non vides de `git diff --cached --name-only`, `unstaged_count` additionne `git diff --name-only` et `untracked_files`.
- `action_diff` : chaque fichier expose `has_secrets_masked` (vrai si le `patch` contient `[REDACTED_SECRET]`).
- `action_release_notes` : valeurs par défaut `from_tag="v0.1.0"`, `to_tag="HEAD"`. Vérification stricte de joignabilité avant toute génération (échec immédiat si injoignable), puis tentative d'inférence LLM avec invite imposant les sections Markdown, et repli sur le formateur déterministe `_format_release_notes_fallback` si le contenu est vide ou inexploitable.
- `_format_release_notes_fallback` : catégorise chaque première ligne de commit via une expression Conventional Commits (`feat|fix|docs|refactor|perf|chore|test|ci|build|revert` `scope` optionnel) en sections `feat`, `fix`, `refactor`, `perf`, `docs`, `other`, avec le SHA court entre backticks ; ajoute la date du jour et `Aucun changement notable.` si aucune section n'est remplie.
- `action_save_config` : simple notification, renvoie toujours un succès sans écrire de fichier.

59.6 Commentaires de fonction (dupliqués du code source)

- `action_status()` : Calcule l'état courant du dépôt Git.
- `action_diff()` : Expose les fichiers stagés avec patchs assainis.
- `action_stage_all()` : Indexe toutes les modifications du dépôt.
- `action_generate_commit()` : Génère un message de commit depuis le diff stagé.
- `action_apply_commit()` : Applique le message de commit validé au dépôt.
- `action_push()` : Pousse les commits locaux vers le dépôt distant.
- `_format_release_notes_fallback()` : Formate les release notes sans appel au LLM.
- `action_release_notes()` : Génère les release notes Markdown entre deux révisions.
- `action_save_config()` : Notifie la prise en compte de la configuration.
- `action_ping_ollama()` : Diagnostique la joignabilité d'Ollama avec modèles installés.
- `main()` : Exécute l'action demandée et affiche le JSON résultat.

---

60 git_wrapper

61 Wrapper Git

Fichier source associé : src/git_commit_release_notes_generator/service/git_wrapper.py

61.1 Description

Ce module encapsule les interactions avec GitPython pour récupérer les diffs, filtrer les secrets et fournir un contrat de données exploitable par la génération de messages de commit.

61.2 Responsabilités

- ouvrir un dépôt Git à partir d'un chemin local ;
- récupérer le diff indexé (`git diff --cached`) ;

- normaliser les sections de diff en objets `DiffFile` ;
- détecter les fichiers binaires et ignorer les diff binaires dans les payloads LLM ;
- masquer les secrets dans le texte avant transmission à l'IA ;
- exposer des payloads structurés pour le reste de l'application.

61.3 Points clés

61.3.1 `GitWrapper`

La classe centrale s'occupe de la lecture du dépôt et du parsing des diff Git. Elle renvoie une liste de `DiffFile`, chaque objet contenant :

- `path` : chemin du fichier modifié ;
- `status` : type de changement (`A`, `M`, `D`, `R`, …) ;
- `added` / `removed` : nombre de lignes modifiées ;
- `is_binary` : indicateur de fichier binaire ;
- `patch` : contenu brut du diff.

61.3.2 `GitWrapperError`

Exception métier utilisée pour signaler un échec de lecture Git ou un problème d'extraction du diff.

61.3.3 `CommitInfo`

Représente un commit de l'historique Git, avec son SHA, message, auteur et date.

61.4 Rôle dans le flux

Le wrapper est le point d'entrée entre le dépôt Git et les composants de génération. Il constitue le contrat technique qui alimente ensuite le module de génération de messages via les objets `DiffFile`.

61.5 Détails déplacés du code source

- `SECRET_PATTERNS` : trois expressions régulières (clés `api[_-]?key|secret|token|password|auth` avec valeur d'au moins 8 caractères, tokens `ghp_` de 36 caractères, JWT `ey...`). `_sanitize_diff_text()` les remplace par `[REDACTED_SECRET]` en conservant le nom du champ capturé quand il existe ; `sanitize_diff()` en est le point d'entrée public.
- `_parse_diff()` : normalise les lignes `diff --git`, découpe par fichier, résout `a/.../b/...` et `/dev/null`. Le statut vaut `A` (marqueur `new file mode` ou ancien chemin `/dev/null`), `D` (marqueur `deleted file mode` ou nouveau chemin `/dev/null`), `R` (renommage), sinon `M`. Les marqueurs de mode ne sont matchés qu'en début de ligne pour ne pas confondre le contenu ajouté/supprimé. Un fichier est binaire si sa section contient `Binary files` ou `GIT binary patch` (compteurs à 0) ; sinon `added`/`removed` comptent les lignes `+`/`-` hors en-têtes `+++`/`---`.
- `get_staged_diff(max_chars=8000)` : lit `git diff --cached --no-color --diff-filter=ACMRTUXBD`, écarte les fichiers binaires, et tronque tout `patch` dépassant `max_chars` avec le marqueur `... [DIFF TRUNCATED: TROP VOLUMINEUX] ...`.
- `get_staged_diff_as_json(max_chars=8000)` : renvoie le diff indexé sous forme de liste de dictionnaires JSON-ready, un par fichier modifié. Chaque dictionnaire est produit par `DiffFile.to_dict()` (dans `models.py`) : `path`, `status`, `added`, `removed`, `is_binary` (booléen réel), `patch`.

- `get_staged_diff_payload(max_chars=8000)` : construit le payload final sans contexte de prompt, directement exploitable par le LLM. Structure : `summary` (`file_count`, `binary_file_count`, `truncated`) + `files` (liste des dictionnaires). `truncated` vaut True si au moins un `patch` a atteint `max_chars`.
- `get_llm_payload(max_chars=8000)` : simple alias de `get_staged_diff_payload()` ; sépare l'intention (« payload pour LLM ») de l'implémentation.
- `get_commits_between_tags(tag_a, tag_b)` : parcourt `tag_a..tag_b` et renvoie des `CommitInfo` (SHA court à 7 caractères, message nettoyé, auteur, date ISO).
- `commit(message)` : refuse si rien n'est indexé (`is_dirty(index=True)` faux) et renvoie le SHA court du commit créé.

61.6 Commentaires de fonction (dupliqués du code source)

- `GitWrapper.__init__()` : Ouvre le dépôt Git à partir du chemin donné.
- `GitWrapper._sanitize_diff_text()` : Masque les secrets détectés dans le texte du diff.
- `GitWrapper.sanitize_diff()` : Expose publiquement le masquage des secrets du diff.
- `GitWrapper._parse_diff()` : Découpe le diff brut en objets DiffFile par fichier.
- `GitWrapper.get_staged_diff()` : Récupère et filtre le diff stagé du dépôt.
- `GitWrapper.get_staged_diff_as_json()` : Expose le diff stagé sous forme de liste JSON.
- `GitWrapper.get_staged_diff_payload()` : Construit le payload résumé du diff stagé.
- `GitWrapper.get_llm_payload()` : Fournit le payload du diff destiné au LLM.
- `GitWrapper.get_commits_between_tags()` : Liste les commits entre deux révisions Git.
- `GitWrapper.commit()` : Crée un commit avec le message donné.

---

62 utils

63 Utilitaires de service

Fichier source associé : src/git_commit_release_notes_generator/service/utils.py

63.1 Description

Ce module réunit les fonctions utilitaires destinées à simplifier le traitement, la validation et le filtrage des données transitant entre le diff Git et la génération de message.

63.2 Rôle attendu

- mettre en forme les données ;
- normaliser les entrées ;
- factoriser les blocs de logique réutilisables ;
- isoler les opérations auxiliaires du cœur métier.

63.3 À compléter

Le fichier est vide à l'état actuel ; ce document sert de point d'entrée pour documenter les utilitaires ajoutés ultérieurement.

---

64 Métriques

65 Script de mesure de conformité et de latence

Fichier source associé : scripts/run_metrics.py

65.1 Description

Outil de développement (pas le module lui-même) qui fait tourner `generate_commit_message()` sur les 6 golden files et affiche un tableau récapitulatif : conformité, correspondance du type attendu, taux de retry, latence.

65.2 Flux de travail

1. Ajoute un handler de logging temporaire pour compter les warnings de retry émis par `llm.py`, sans modifier son API publique.
2. Pour chaque golden file : chronomètre l'appel à `generate_commit_message()`, capture le résultat ou l'échec de validation.
3. Affiche un tableau ligne par ligne (scénario, résultat, type attendu/obtenu, retry, latence) puis un récapitulatif chiffré.

65.3 Points clés

- écrit volontairement sur stdout : ce n'est pas le module de production, qui lui doit rester silencieux ;
- nécessite un accès réseau au serveur Ollama partagé, aucun mock ;
- sert de base chiffrée pour la soutenance (taux de conformité, latence moyenne).

65.4 Dépendances

- llm.md
- golden_diffs.md

65.5 Détails déplacés du code source

- Usage : `python scripts/run_metrics.py`. `_RetryCounter` compte les `warning` contenant `invalide` émis par `llm.py` pour détecter les générations ayant nécessité un retry, sans toucher à son API publique.
- `_print_report()` affiche le tableau (`Scénario`, `Résultat`, `Attendu`, `Obtenu`, `Retry`, `Latence`) puis les taux de conformité, de type exact, de retry et les latences moyenne/min/max.

65.6 Commentaires de fonction (dupliqués du code source)

- `_RetryCounter.__init__()` : Initialise le compteur de retries détectés.
- `_RetryCounter.emit()` : Compte les avertissements signalant une génération invalide.
- `main()` : Mesure la conformité et la latence sur les diffs de référence.
- `_print_report()` : Affiche le tableau récapitulatif des mesures.

---

66 PDF

67 Script de génération du PDF de documentation

Fichier source associé : scripts/generate_documentation_pdf.py

67.1 Description

Ce script assemble l'ensemble des pages Markdown référencées dans la navigation de `mkdocs.yml` en un document unique, puis le convertit en PDF via `pandoc` et `xelatex`.

67.2 Flux de travail

1. `collect_pages()` parcourt récursivement la clé `nav` de `mkdocs.yml` (dictionnaires et listes) et collecte les couples `(titre, chemin relatif)` sans doublon.
2. `build_markdown()` lit chaque page (chemin relatif à la racine, avec repli vers `docs/`), préfixe son contenu par un titre `# {titre}`, concatène le tout après une table des matières (`\tableofcontents`), et écrit le résultat dans `.tmp-doc/documentation.md`.
3. `main()` vérifie la présence de `pandoc` et `xelatex` (message d'installation `apt-get` sinon), puis lance `pandoc` avec les options `--from=markdown+raw_tex --standalone --toc --number-sections --pdf-engine=xelatex`, des marges de 1 pouce, des liens colorés en bleu et les polices `DejaVu Sans` / `DejaVu Sans Mono`, vers `documentation.pdf` à la racine.

67.3 Points clés

- Toute page référencée mais introuvable lève `FileNotFoundError` avec le chemin fautif.
- Le script ne dépend que de `pathlib`, `shutil`, `subprocess` et `yaml`.

67.4 Commentaires de fonction (dupliqués du code source)

- `collect_pages()` : Collecte les pages de documentation sans doublon.
- `build_markdown()` : Assemble les pages Markdown en document unique.
- `main()` : Vérifie les outils et génère le PDF de documentation.

---

68 CLI Go (guide)

69 Architecture et Fonctionnement de l'Interface CLI (Go)

69.1 Guide Pratique : Compiler, Tester et Présenter

69.1.1 Compiler le projet :

Dans un terminal, placez-vous dans le dossier `src/cli` :

```
cd SAE_sujet_8_application_intelligente/src/cli
go build -o git-generator main.go
```

69.1.2 Options de la ligne de commande (Flags)

Le binaire propose les options suivantes pour s'adapter à tous les contextes d'utilisation :

Option	Valeur par défaut	Description
`--repo <chemin>`	`.` (répertoire courant)	**Dépôt Git cible à analyser** : chemin du projet sur lequel vous travaillez (pour extraire le `git diff`, générer un message de commit ou créer les release notes).
`--dev`	`false`	**Mode développeur** : débloque l'utilisation des drapeaux de mock en CLI et autorise l'affichage et la modification des réglages de mock dans l'écran de configuration.
`--mockInterface`	`false`	**Mock de l'interface** : force le `MockClient` pour manipuler et tester l'interface TUI hors-ligne sans sous-processus Python (requiert obligatoirement `--dev`).
`--mockIA`	`false`	**Mock de l'IA** : active la simulation des réponses Ollama (commits et release notes déterministes hors-ligne, requiert obligatoirement `--dev`).
`--app-dir <chemin>`	Auto-détecté	**Racine de l'application Gitelligence** : chemin vers le dossier de notre outil (`SAE_sujet_8_application_intelligente`) où résident le fichier `.env` applicatif et le code Python.

[!CAUTION] **Contrôle d'accès au mode mock**
L'utilisation de `--mockInterface` ou `--mockIA` sans le flag `--dev` lève une erreur bloquante et interrompt immédiatement l'exécution.

[!NOTE] **Pourquoi séparer `--repo` et `--app-dir` ? (Isolation stricte)**
Dans une utilisation classique, l'outil analyse un projet externe (ex: `--repo ~/projets/mon-site`). La distinction garantit que : 1. Les réglages de notre application (URL Ollama, modèle, timeout, etc.) sont lus et modifiés **exclusivement** dans le `.env` de Gitelligence (`--app-dir`). 2. Le projet cible analysé (`--repo`) n'est **jamais parasité** par la création d'un `.env` applicatif.

Dans 99% des cas, `--app-dir` est **détecté automatiquement** par l'application (en inspectant le chemin du binaire ou du code source). Ce drapeau n'est utile que si vous installez `git-generator` dans un répertoire système global (ex. `/usr/local/bin`) et que vous l'exécutez depuis un tout autre dossier.

69.1.3 Lancer l'application :

```
# 1. Lancement standard dans le dépôt courant (mode réel ou mock selon le .env) :
./git-generator

# 2. Lancement en mode développeur (accès à la configuration des mocks) :
./git-generator --dev

# 3. Lancement avec mock de l'interface (autonome, requiert --dev) :
./git-generator --dev --mockInterface

# 4. Lancement avec mock de l'IA (requiert --dev) :
./git-generator --dev --mockIA

# 5. Lancement en ciblant un autre dépôt Git :
./git-generator --repo /chemin/vers/un/autre/projet

# 6. Lancement avec indication explicite de la racine de l'application :
./git-generator --repo /chemin/vers/projet --app-dir /chemin/vers/SAE_sujet_8_application_
```

69.1.4 Exécuter les tests unitaires Go :

Conformément à l'architecture du projet, tous les tests sont isolés dans le dossier `test/` :

```
cd test/cli
go test -v ./...
```

69.2 Pourquoi avoir choisi Go pour le Front-end ?

- **Vitesse et instantanéité** : Go se compile directement en code machine natif. L'exécutable se lance en moins de 5 millisecondes, sans aucun temps de démarrage d'interpréteur (comme Python ou Node.js).
- **Binaire unique et autonome** : Aucun besoin d'installer de runtime Go sur la machine cible. Le binaire embarque tout.
- **Maîtrise complète du Terminal** : Possibilité de créer une interface TUI (Terminal User Interface) élégante, fluide et animée sans aucune bibliothèque tierce lourde.

69.3 Comment Go communique avec Python (Le Pont / Bridge)

Toute la communication repose sur le paquet `src/cli/bridge/`, structuré autour de l'interface Go idiomatique `BackendClient`.

69.3.1 Le mécanisme d'appel système (`exec.Command`)

Go n'utilise pas de socket réseau ou de serveur HTTP local pour parler à Python (ce qui serait lourd et risquerait des conflits de ports).
Go lance simplement Python en sous-processus système (implémenté dans `python_client.go`) :

```
// Exemple simplifié de ce que fait Go sous le capot :
cmd := exec.Command("python3", "-m", "git_commit_release_notes_generator.service.core", "-
cmd.Dir = repoPath // S'exécute à la racine du dépôt
```

```
var stdout, stderr bytes.Buffer
cmd.Stdout = &stdout
cmd.Stderr = &stderr

err := cmd.Run()
```

69.3.2 Le protocole JSON et la règle de `stdout` vs `stderr`

Pour que le pont fonctionne sans faille, nous avons fixé une règle universelle : 1. **`stdout` (Sortie Standard)** : Réservé **exclusivement** au JSON de données. Python ne doit JAMAIS faire de `print("chargement...")` sur `stdout`. 2. **`stderr` (Sortie d'Erreur)** : Utilisé pour les logs, les traces de débogage ou les messages d'avertissement de Python.

Dès que la commande Python se termine, Go désérialise le JSON brut reçu sur `stdout` directement dans une structure Go typée grâce à `json.Unmarshal` :

```
var diffResp models.DiffResponse
err := json.Unmarshal(stdout.Bytes(), &diffResp)
// Maintenant, Go manipule des variables fortement typées : diffResp.Files, etc.
```

69.3.3 Architecture du Mock & Inversion des Dépendances (DIP)

Afin de respecter les conventions Go (*"Accept interfaces, return structs"*) et le principe de responsabilité unique (SRP), le pont sépare strictement les contrats, les appels réels et la simulation :

```
classDiagram
    class BackendClient {
        <<interface>>
        +GetRepoStatus() (*RepoStatus, error)
        +GetDiff() (*DiffResponse, error)
        +StageAll() (*ActionResult, error)
        +GenerateCommit(feedback string) (*CommitProposal, error)
        +ApplyCommit(message string) (*ActionResult, error)
        +Push() (*ActionResult, error)
        +GetReleaseNotes(fromTag, toTag string) (*ReleaseNotesResponse, error)
        +GetConfig() (*ConfigResponse, error)
        +SaveConfig(url, model, timeout, lang, mock) (*ActionResult, error)
        +PingOllama() (*PingResponse, error)
        +IsMock() bool
    }

    class BridgeClient {
        -isMock bool
        -realClient *PythonClient
        -mockClient *MockClient
        +activeDelegate() BackendClient
    }

    class PythonClient {
        -pythonBin string
        -moduleName string
        +runPythonCommand()
    }
```

```
    class MockClient {
        +SimulateDelay bool
        +CustomDiff *DiffResponse
        +CustomError error
    }

    BackendClient <|.. BridgeClient
    BackendClient <|.. PythonClient
    BackendClient <|.. MockClient
    BridgeClient --> PythonClient : délègue si réel
    BridgeClient --> MockClient : délègue si simulation
```

69.3.3.1 Configuration via le fichier `.env` (`MOCK_INTERFACE` et `MOCK_AI`) L'activation des mocks est pilotée par les variables `MOCK_INTERFACE` et `MOCK_AI` dans le fichier `.env` ou par les drapeaux `--mockInterface` et `--mockIA` en ligne de commande (sous réserve d'activer le flag `--dev`) :

```
# Dans le .env à la racine de l'application (et .env.exemple)
MOCK_INTERFACE=false  # -> Mode standard : délègue au backend Python (true pour simulation
MOCK_AI=true          # -> Active le mock Ollama (commits et notes déterministes sans serv
```

Lorsque `MOCK_INTERFACE=false`, le client communique avec le sous-processus Python et remonte fidèlement les données ou erreurs réelles.

69.3.3.2 Implémentation du mode mock interface (`mock_client.go`) Le `MockClient` implémente `BackendClient` en fournissant des réponses cohérentes avec les cas de test du projet (secrets masqués, détection binaire, latence de traitement).
Pour les tests unitaires automatisés situés dans le dossier `test/` (`test/cli/bridge_test.go`), il permet de désactiver les délais (`SimulateDelay = false`) et de surcharger les retours ou erreurs (`CustomError`, `CustomDiff`).

69.3.3.3 La bascule dynamique sans recompilation La fonction factory `NewBackendClientWithAppRoot(repoRoot, appRoot, forceMockInterface, forceMockIA)` instancie le coordinateur `BridgeClient`.
Lorsque l'utilisateur modifie la configuration depuis l'écran `[3]` de la CLI (en mode développeur `--dev`) : 1. `SaveConfig` met à jour le fichier `.env` à la racine de l'application via le module dédié `env.go`. 2. Le `BridgeClient` bascule instantanément son délégué actif entre `MockClient` et `PythonClient` sans nécessiter **la moindre modification ni recompilation de code côté Go**.

69.4 Architecture Interne du Code Go (`src/cli/`)

69.4.1 Arborescence détaillée

```
src/cli/
├── go.mod                    # Définition du module Go (go 1.22, 0 dépendance externe)
├── main.go                   # Point d'entrée, capture Ctrl+C, boucle du menu
├── models/
│   └── types.go              # Structs Go typées mappées sur les schémas JSON
├── bridge/
│   ├── client.go             # Interface BackendClient et coordinateur BridgeClient
│   ├── python_client.go      # Implémentation réelle (exec.Command & unmarshal JSON)
```

```
│   ├── mock_client.go         # Implémentation simulation / mock pour mode démo & tests
│   └── env.go                 # Gestionnaire autonome du fichier .env (lecture/écriture)
├── ui/
│   ├── styles.go              # Codes ANSI, TrueColor, calcul VisualLen, StripANSI
│   ├── box.go                 # Tracé des fenêtres, bordures Unicode, tableaux
│   └── spinner.go             # Goroutine de chargement dynamique avec chronomètre
└── views/
    ├── menu_view.go           # Écran 1 : Accueil et affichage du contexte dépôt
    ├── commit_view.go         # Écran 2 : Diff, validation, feedback LLM, git push
    ├── release_notes_view.go  # Écran 3 : Sélection tags et rendu Markdown
    └── config_view.go         # Écran 4 : Paramètres .env et test ping Ollama
```

69.4.2 Rôle de chaque paquet :

- **`models`** : Ne contient aucune logique. Seulement les définitions de structures (`RepoStatus`, `DiffFile`, `CommitProposal`, `ConfigResponse`, etc.).
- **`bridge`** : Expose l'interface `BackendClient` et orchestre les communications avec Python ou le mock.
- **`ui`** : La boîte à outils graphique. Gère l'affichage à l'écran, les couleurs, les bordures et les animations.
- **`views`** : Contient les 4 écrans interactifs. Chaque vue prend en paramètre le `bufio.Reader`, l'interface `BackendClient` et le `RepoStatus`.

69.5 Fonctionnement Écran par Écran

69.5.1 Écran 1 : Menu Principal & Contexte Dépôt

- Dès l'ouverture, Go appelle `client.GetRepoStatus()`.
- Python lui répond en JSON :

  ```
  { "is_git_repo": true, "branch": "interface-cli", "staged_count": 3, "unstaged_count":
  ```

- Go génère la bannière d'en-tête avec les pastilles de couleur :
  - Si `staged_count > 0` : affiché en vert.
  - Si `staged_count == 0` : affiché en jaune avec avertissement.
  - Indication discrète du mode uniquement si la simulation est active : `Mode simulation actif (MOCK_INTERFACE=true dans le .env)` (aucun message superflu en mode réel standard).
- L'utilisateur tape 1, 2, 3, 4 ou q pour naviguer.

69.5.2 Écran 2 : Génération de Commit & Boucle de Décision

C'est le cœur de l'outil :

1. **Vérification du diff** : Go demande les fichiers modifiés (`client.GetDiff()`).
2. **Auto-stage** : Si aucun fichier n'est indexé, Go demande :
   `Voulez-vous indexer automatiquement toutes les modifications (git add -A) ? [O/n]`
   Si oui, Go demande à Python d'exécuter l'ajout, puis réactualise le diff.
3. **Tableau visuel** : Affiche les fichiers modifiés :
   - `[M]` en cyan pour modifié.
   - `[A]` en vert pour ajouté.

- `[D]` en rouge pour supprimé.
- Badge spécial `[Secret Masqué]` si Dorian a détecté une clé API ou un token dans le patch.
- Badge `[Binaire]` pour les images ou exécutables exclus.

4. **Appel IA** : Le spinner tourne pendant que le modèle `gemma4:12b` réfléchit.
5. **Carte Conventional Commits** : Affichage encadré du résultat structuré :
   - Type (`feat`, `fix`, `docs`, etc.)
   - Scope (`auth`, `cli`, etc.)
   - `Subject` (la phrase de commit)
   - Body (explications complémentaires)
6. **Boucle d'actions** :
   - **`[V] Valider`** : Go demande à Python d'appliquer le commit (`client.ApplyCommit`). Python exécute `git commit` et renvoie le SHA (ex: `7a9e2f4`). Go demande ensuite si l'utilisateur souhaite faire un `git push`.
   - **`[R] Régénérer avec consigne`** : Go demande un texte à l'utilisateur (ex: *"Sois plus concis"* ou *"C'est plutôt un fix"*). Go renvoie ce feedback à Python pour réinterroger le LLM.
   - **`[M] Modifier manuellement`** : Permet de retaper directement le sujet ou le body dans la console avant de valider.
   - **`[A] Annuler`** : Revient au menu sans rien toucher.

69.5.3 Écran 3 : Génération des Release Notes

1. Go demande à l'utilisateur :
   - Le tag de départ (par défaut `v0.1.0`).
   - Le tag d'arrivée (par défaut `HEAD`).
2. Python extrait les commits entre ces deux bornes et demande au LLM de rédiger un changelog structuré.
3. Go affiche le document Markdown dans une boîte dédiée.
4. L'utilisateur peut appuyer sur `[S]` pour enregistrer automatiquement le résultat dans un fichier `RELEASE_NOTES.md` à la racine du dépôt.

69.5.4 Écran 4 : Configuration (.env) & Diagnostic Ping Ollama

Cet écran répond directement aux **consignes impératives de contrôle de l'IUT** : * Affiche les variables lues par Python et Go : * `OLLAMA_BASE_URL` (par défaut `http://10.22.28.190:11434`) * `OLLAMA_MODEL` (par défaut `gemma4:26b`) * `OLLAMA_TIMEOUT_S` (`300s`) * `MOCK_INTERFACE` et `MOCK_AI` (visibles et modifiables uniquement si le mode `--dev` est actif) * **Action `[T]` (Test Ping)** : Go demande à Python d'effectuer une requête rapide vers le serveur Ollama. S'il répond, l'écran affiche en vert la latence en millisecondes et la liste réelle des modèles installés sur le serveur ! * **Action `[M]` (Modification)** : Permet de modifier l'adresse IP, le modèle, le timeout (et d'activer/désactiver les mocks si `--dev` est actif). Go sauvegarde immédiatement les valeurs dans le fichier `.env` (✔ `Configuration sauvegardée dans .env.`).
 Aucune recompilation du binaire Go n'est nécessaire.

---

70 Présentation

71 Présentation des tests

Cette section décrit la documentation technique associée aux tests du projet.

71.1 Objet

Le dossier `test/` contient les tests unitaires, d'intégration et de situation. Cette documentation permet de comprendre le rôle de chaque test et la stratégie de validation du projet.

71.2 Types de tests présents

- tests unitaires : vérification des fonctions et composants isolés
- tests de wrapper Git : validation de l'interaction avec Git
- tests Ollama LLM : vérification des appels et réponses du modèle
- golden diffs : tests sur des sorties attendues
- tests de situation : scénarios réalistes du projet

71.3 Organisation

La structure des fichiers de documentation suit celle du dossier `test/` pour faciliter la lecture et la maintenance.

71.4 Rôle de cette documentation

Chaque test ou groupe de tests possède une page dédiée afin de :

- expliquer le but du test
- détailler le scénario couvert
- documenter les résultats attendus
- faciliter le suivi qualité du projet

71.5 À retenir

Les tests sont une partie essentielle de la fiabilité du projet. Leur documentation doit rester à jour pour refléter les cas de validation réellement mis en place dans le code.

---

72 Core (point d'entrée)

73 Tests unitaires du point d'entrée Core

Fichier source associé : test/test_core.py

73.1 Description

Ce fichier de test valide le comportement du module `git_commit_release_notes_generator.service.core` garantissant la conformité avec le contrat d'échange JSON du front-end Go.

73.2 Cas de tests couverts

1. **test_action_status_on_valid_repo** : vérifie la conformité de l'objet d'état sur un vrai dépôt Git (branche, compteurs staged / unstaged).
2. **test_action_status_on_non_git_repo** : vérifie la gestion propre des erreurs sur un dossier hors-Git (`is_git_repo = False`).
3. **test_action_diff_structure** : vérifie la structure de chaque fichier de diff (`path`, `status`, `binary`, `patch`, `has_secrets_masked`).
4. **test_action_generate_commit_no_staged_files** : vérifie le retour d'erreur lorsqu'aucun fichier n'est indexé.
5. **test_action_generate_commit_success** : simule une inférence LLM réussie avec serveur joignable et intégration du `feedback` utilisateur.
6. **test_action_generate_commit_ollama_unreachable** : vérifie l'échec immédiat et explicite si Ollama est injoignable (`check_ollama_reachable = False`), sans tenter l'appel de génération.
7. **test_action_generate_commit_network_error_fails_fast** : vérifie le comportement de bout en bout lorsqu'une erreur réseau urllib survient lors du ping.
8. **test_action_apply_commit_validation** : rejette les messages de commit vides.
9. **test_action_apply_commit_success** : simule l'application d'un commit Git et vérifie la présence du SHA.
10. **test_action_push_no_remote** : gère l'absence de remote Git configuré sans plantage.
11. **test_action_release_notes_formatting** : teste le formateur déterministe Conventional Commits (catégorisation Features, Fixes, Docs) lorsque le ping réussit mais que le chat LLM échoue.
12. **test_action_release_notes_ollama_unreachable** : vérifie que la génération des release notes échoue immédiatement si le serveur Ollama est injoignable au ping (`check_ollama_reachable = False`), à l'identique des commits.
13. **test_action_release_notes_empty** : vérifie la réponse lors d'un historique vide entre deux bornes identiques.
14. **test_unknown_action_exits_zero_with_error_json** : vérifie que le process sort avec le code 0 et renvoie une erreur JSON propre sur action inconnue.
15. **test_action_save_config** : vérifie la prise en compte de la notification de configuration.
16. **test_action_ping_ollama_reachable** : simule une réponse positive d'Ollama avec latence et modèles installés.
17. **test_action_ping_ollama_unreachable** : vérifie le retour propre en cas de défaillance réseau (`reachable = False`).

---

74 Git wrapper

75 Tests du wrapper Git

Fichier source associé : test/test_git_wrapper.py

75.1 Description

Ce module valide le comportement du wrapper Git utilisé pour analyser les diffs et préparer les données envoyées au système de génération de commit.

75.2 Ce qui est testé

- le parsing d'un diff Git en fichiers structurés ;

- la détection des modifications par fichier ;
- la suppression des données sensibles dans les patches ;
- la création d'un payload JSON exploitable par le modèle ou par l'orchestrateur applicatif.

75.3 Cas de test principaux

75.3.1 1. Parsing d'un diff multi-fichiers

Le test vérifie que le diff est découpé correctement en plusieurs objets, avec pour chaque fichier :

- le chemin du fichier ;
- le statut de modification ;
- les lignes ajoutées et supprimées ;
- le patch complet associé.

75.3.2 2. Sanitisation des secrets

Le test simule un contenu contenant une clé ou une valeur sensible et vérifie que le mécanisme de redaction remplace bien les informations sensibles par une valeur de type `[REDACTED_SECRET]`.

75.3.3 3. Structure du payload JSON

Le test s'assure que les objets renvoyés par le wrapper sont convertibles en payloads normalisés, sans informations parasites ou inutiles. Il vérifie notamment la présence des champs attendus comme `path`, `status`, `patch` et `is_binary`.

75.4 Points de vigilance

- La logique de parsing doit rester tolérante aux variations de format de diff Git.
- La redaction des secrets doit être systématique avant toute transmission hors du contexte local.
- Les payloads doivent rester structurés et légers pour éviter d'encombrer le modèle LLM.

---

76 Intégration Git wrapper (slow)

77 Test d'intégration réel avec le wrapper Git

Fichier source associé : test/test_integration_git_wrapper_slow.py

77.1 Description

Ce test vérifie que le `list[DiffFile]` réellement produit par `GitWrapper.get_staged_diff()` (module Git) est directement consommable par `generate_commit_message()` (module LLM), sans conversion manuelle entre les deux. Il s'appuie sur les 12 vrais dépôts Git de `test/situation_test/commit/` plutôt que sur des fixtures écrites à la main.

77.2 Ce qui est testé

- que le wrapper Git et le client LLM communiquent sans erreur de compatibilité de contrat sur 8 scénarios réels (feat, fix, docs, refactor, test, chore, perf, build) ;
- que chaque scénario produit un `CommitMessage` avec un type et un subject non vides.

77.3 Stratégie de test

Avant chaque scénario, le test indexe (`git add -A`) les modifications locales du dépôt factice correspondant, puisque `get_staged_diff()` ne lit que le diff indexé (`git diff --cached`). Nécessite `GitPython` installé, les fixtures activées (`python test/situation_test/init_test.py`), et un accès réseau au serveur Ollama partagé. Marqué `slow`, exclu par défaut. Compte 5 à 10 minutes pour les 8 scénarios selon la charge du serveur.

77.4 Cas importants

77.4.1 compatibilité de contrat entre deux modules

Le test ne compare pas d'égalité stricte de type : son but premier est de prouver qu'aucune erreur (attribut manquant, format inattendu) ne survient quand un vrai `DiffFile` du module Git traverse le module LLM.

77.5 Points de vigilance

- Les dépôts de `situation_test/` contiennent plusieurs fichiers modifiés à la fois (code, tests, CI, doc), donc des diffs plus volumineux que les golden files : latence et probabilité de timeout plus élevées, surtout si le serveur partagé est chargé.
- Toujours désactiver les fixtures après usage (`python test/situation_test/deinit_test.py`) pour ne pas laisser de vrais `.git` actifs dans l'arborescence suivie par le dépôt principal.

---

78 Ollama LLM

79 Tests du client Ollama

Fichier source associé : test/test_ollama_llm.py

79.1 Description

Ce module couvre la logique de validation et d'interaction avec le client Ollama. Il vérifie notamment le comportement du code qui transforme un diff en un message de commit conforme à la norme Conventional Commits.

79.2 Ce qui est testé

- la génération d'un message de commit valide dès le premier essai ;
- le mécanisme de retry lorsqu'une réponse est invalide ;
- l'échec après plusieurs réponses invalides ;
- les règles de validation du message (`type`, `scope`, `subject`) ;

- la vérification de joignabilité rapide et extraction des modèles via `check_ollama_reachable()` / `ping_ollama()` (succès, échec réseau, mesure de latence, récupération des modèles avec `fetch_models`, sérialisation `to_dict`, compatibilité tuple) ;
- la compatibilité avec les anciennes structures de données binaires ;
- l'override déterministe du type via `feedback` : une consigne contenant une formulation d'intention explicite ("c'est un fix, je veux un fix") force le type final même si le modèle en propose un autre ; une consigne sans mot-clé de type, ou avec deux types contradictoires, ne déclenche aucun override ; la simple présence du mot "test" (homonyme d'un type et d'un mot français courant) sans formulation d'intention ne doit pas non plus déclencher d'override ; `_extract_forced_type()` est aussi testée directement, isolément de la génération.

79.3 Stratégie de test

Les appels réseau réels sont évités par remplacement de `urllib.request.urlopen` avec un faux objet de réponse. Cela permet de simuler le comportement de Ollama sans dépendre d'un service distant.

79.4 Cas importants

79.4.1 génération réussie au premier essai

Le test vérifie qu'un payload bien formé est accepté et transformé en objet métier avec les bons champs : type, scope et subject.

79.4.2 retry puis succès

Lorsque la première réponse est invalide, le système réessaie avec une seconde réponse. Cela confirme que la logique de récupération est bien mise en place.

79.4.3 validation stricte du message

Le test contrôle que les sujets terminés par un point, ainsi que les types inconnus, sont rejetés pour rester conforme à la convention de messages attendue.

79.5 Points de vigilance

- La validation doit rester stricte pour éviter des messages de commit peu lisibles ou incohérents.
- Le mécanisme de retry ne doit pas masquer des erreurs de fond ou des réponses non exploitables.
- La compatibilité avec les objets historiques de type `binary` doit être conservée pour ne pas casser les données existantes.
- La détection d'un type imposé par `feedback` doit rester prudente : exiger une formulation d'intention explicite, pas la simple présence du mot, sinon toute mention anodine d'un mot comme "test" casserait le type choisi par le modèle.

---

80 Ollama LLM (slow, réseau réel)

81 Test réseau réel du client Ollama

Fichier source associé : test/test_ollama_llm_slow.py

81.1 Description

Ce test vérifie que le client Ollama fonctionne réellement contre le serveur partagé, et pas seulement contre une réponse simulée. Il est marqué `slow` et n'est jamais exécuté par défaut.

81.2 Ce qui est testé

- un appel complet `generate_commit_message()` sur un diff réaliste (ajout d'une fonction de connexion) ;
- que le serveur Ollama répond, que la réponse est bien un JSON conforme, et que le `CommitMessage` obtenu a un `type` et un `subject` non vides.

81.3 Stratégie de test

Aucun mock : le test appelle vraiment `http://10.22.28.190:11434`. Il nécessite donc un accès réseau au serveur partagé et peut prendre 15 à 45 secondes selon que le modèle est déjà chargé en mémoire côté serveur.

81.4 Cas importants

81.4.1 génération contre le vrai modèle

Le test affiche (`-s`) le `type`, le `scope`, le `subject` et le `body` obtenus, pour une inspection visuelle rapide en plus des assertions automatiques.

81.5 Points de vigilance

- Ce test doit rester exclu de l'exécution par défaut (`pytest.ini` filtre déjà `not slow`) pour ne pas ralentir ni faire échouer la suite rapide en cas de coupure réseau.
- Une latence de 15 à 45 secondes est normale, pas un signe de bug.

---

82 Mock IA

83 Tests du Mock IA (Simulation Hors-Ligne)

Fichier source associé : `test/test_mock_ai.py`

83.1 Description

Ce fichier contient la suite de tests automatisés validant le module de simulation de l'intelligence artificielle (`mock_llm.py`), l'interprétation des variables de configuration d'environnement, et la bascule dynamique via la variable `MOCK_AI`.

83.2 Ce qui est testé

1. **`test_parse_bool`** : vérifie la robustesse du parsing des valeurs booléennes issues de l'environnement (`true`, `TRUE`, `1`, `yes`, `oui`, `false`, `0`, etc.).
2. **`test_mock_generate_commit_empty_diff_raises`** : valide qu'une exception `ValueError` est levée lorsqu'aucun diff n'est fourni.
3. **`test_mock_generate_commit_static_message`** : vérifie la conformité Conventional Commits du message statique généré (`feat(mock): commit mocker pour test hors ligne`).
4. **`test_mock_generate_commit_with_user_feedback`** : contrôle l'injection de la consigne utilisateur dans le corps du message lors d'une révision.
5. **`test_mock_call_chat`** : vérifie la structure de la réponse JSON simulée de l'API de chat Ollama.
6. **`test_mock_release_notes`** : confirme la génération d'une note de version Markdown statique déterministe.
7. **`test_mock_ping`** : contrôle la simulation d'un ping réussi vers Ollama avec latence factice et liste de modèles installés.
8. **`test_llm_check_ollama_reachable_when_mock_ai`** : valide que le ping rapide répond immédiatement sans requête HTTP lorsque `MOCK_AI=True`.
9. **`test_llm_generate_commit_delegates_to_mock_when_mock_ai`** : valide l'aiguillage automatique de `generate_commit_message` vers le mock.
10. **`test_core_action_generate_commit_with_mock_ai`** : valide le fonctionnement de l'action `generate-commit` du service core sous mode mock.
11. **`test_core_action_release_notes_with_mock_ai`** : valide l'action `release-notes` du service core sous mode mock.
12. **`test_core_action_ping_ollama_with_mock_ai`** : valide l'action `ping-ollama` du service core sous mode mock.
13. **`test_llm_generate_commit_mock_ai_feedback_forces_type`** : vérifie que l'override déterministe de type s'applique aussi en mode mock — le mock renvoie toujours `type="feat"` codé en dur, mais un feedback citant explicitement "fix" doit tout de même forcer le résultat final à `fix`.

83.3 Stratégie de test

Les tests s'exécutent entièrement hors-ligne sans nécessiter de serveur Ollama ni d'accès réseau. Ils s'appuient sur `unittest.mock.patch` pour isoler `config.MOCK_AI` et valider les deux branches d'exécution.

83.4 Exécution des tests

```
pytest test/test_mock_ai.py
```

---

84 Golden diffs

85 Fixtures Golden Diff

Fichier source associé : test/fixtures/golden_diffs.py

85.1 Description

Ce module fournit des scénarios de diff déjà préparés, appelés `GoldenDiff`, pour simuler des changements de développement classiques et tester le bon classement des modifications par type de commit.

85.2 Structure de données

Chaque fixture contient :

- un `name` de scénario ;
- un `expected_type` correspondant au type attendu (`feat`, `fix`, `docs`, `refactor`, `test`, etc.) ;
- une liste de `DiffFile` représentant les fichiers modifiés.

85.3 Rôle fonctionnel

Ces fixtures servent de références pour valider la capacité du système à reconnaître la nature d'un changement à partir de son diff, sans dépendre d'un contexte réel de développement.

85.4 Exemple de scénarios

Parmi les cas couverts, on trouve :

- une fonctionnalité ajoutée (feature) ;
- une correction de bug ;
- une documentation mise à jour ;
- un refactoring ;
- l'ajout de tests ;
- une modification de dépendances ou de build.

Trois scénarios volontairement ambigus ont été ajoutés pour sonder les limites réelles du prompt (cf. carte Trello #34, « l'IA confond parfois fix/feat ») :

- `fix_disguised_as_bounds_check` : une validation ajoutée qui corrige en réalité un bug (prix négatif possible), pour vérifier que le modèle privilégie bien "fix" même quand le diff a l'air d'ajouter une capacité ;
- `chore_add_makefile_lint_target` : une modification de Makefile, pour vérifier que le modèle ne la confond pas avec "build" (réservé au packaging/dépendances selon le prompt) ;
- `refactor_hides_rounding_fix` : une extraction de fonction qui corrige au passage un bug d'arrondi, pour vérifier que le modèle priorise "fix" sur "refactor" quand les deux sont mélangés.

Mesuré via `scripts/run_metrics.py` sur le vrai serveur Ollama : 100% de conformité et de type exact sur les 9 scénarios (les 6 historiques + ces 3 nouveaux), un seul retry observé (subject 1 caractère trop long, corrigé automatiquement).

85.5 Points de vigilance

- Les scénarios doivent rester représentatifs des modifications réelles du projet.
- Les types attendus servent de repères de validation, sans remplacer la logique métier du modèle.
- Les fixtures doivent rester stables pour assurer des tests reproductibles.

---

86 Golden diffs (slow, réseau réel)

87 Test réseau réel sur les golden files

Fichier source associé : test/test_golden_diffs_slow.py

87.1 Description

Ce test paramétré fait tourner `generate_commit_message()` contre le vrai serveur Ollama, une fois par golden file (`test/fixtures/golden_diffs.py`). Il vérifie que le client reste correct sur plusieurs types de changement représentatifs (feat, fix, refactor, docs, test, build), pas seulement sur un cas isolé.

87.2 Ce qui est testé

- que chaque scénario produit un `CommitMessage` valide (type et subject non vides) ;
- affichage du type attendu vs obtenu et de la latence, pour repérer les divergences de jugement du modèle sans faire échouer le test pour autant (le type exact n'est pas une assertion stricte, car la génération n'est pas déterministe).

87.3 Stratégie de test

Aucun mock, marqué `slow`, exclu par défaut. Nécessite un accès réseau au serveur Ollama partagé. Compte environ 3 minutes pour les 6 scénarios.

87.4 Cas importants

87.4.1 comparaison type attendu / type obtenu

Chaque scénario affiche une ligne `[nom_scenario] attendu=X obtenu=Y en Zs`, utile pour suivre à l'œil les divergences (ex. `chore` vs `build`) sans avoir besoin du script de métriques.

87.5 Points de vigilance

- Une divergence de type n'est pas forcément un bug : certaines catégories Conventional Commits (`chore`/`build`) sont ambiguës par nature.
- Ce test ne fait qu'observer ; c'est `scripts/run_metrics.py` qui calcule un taux agrégé sur l'ensemble des scénarios.

---

88 Bridge

89 Tests Unitaires du Bridge Go

Fichier source associé : test/cli/bridge_test.go

89.1 Description

Ce fichier contient la suite de tests automatisés validant le pont Go ↔ Python, la gestion du fichier `.env`, le basculement dynamique Mock/Réel, et l'isolation stricte des répertoires.

89.2 Tests couverts

1. **`TestMockClientBasics`** : vérifie les réponses par défaut du `MockClient` (statut du dépôt, diff avec secrets masqués, message de commit, release notes).
2. **`TestMockClientOverrides`** : valide l'injection de réponses personnalisées et d'erreurs sur mesure pour tester la robustesse des vues.
3. **`TestParseBoolFromString`** : vérifie le parsing des booléens textuels issus du fichier `.env` (`true`, `false`, `1`, `0`, `yes`, `oui`, etc.).
4. **`TestEnvFileReadWrite`** : valide l'écriture et la relecture d'un fichier `.env` temporaire.
5. **`TestBridgeClientSwitching`** : vérifie la bascule dynamique entre `MockClient` et `PythonClient` lors de l'appel à `SaveConfig`.
6. **`TestPythonClientRealIntegration`** : valide l'exécution réelle du sous-processus Python (`--action status`, `diff`, `ping-ollama`).
7. **`TestAppEnvIsolationFromTargetRepo`** : garantit que lors de l'analyse d'un dépôt cible externe, le `.env` est créé/modifié **uniquement** dans l'application et **jamais** dans le dépôt cible.
8. **`TestAppEnvNeverInSrc`** : vérifie formellement que l'application refuse d'écrire dans `src/` et que le fichier `.env` réside obligatoirement à la racine du projet.
9. **`TestSaveConfigBothMocks`** : valide l'enregistrement conjoint et la relecture distincte des réglages `MOCK_INTERFACE` et `MOCK_AI` dans le fichier `.env`.

89.3 Exécution des tests

```
cd test/cli
go test -v ./...
```

---

90 Module Go

91 Module Go de Test

Fichier source associé : test/cli/go.mod

91.1 Description

Ce fichier définit le module Go secondaire (`sae-git-cli-test`) utilisé pour exécuter les tests unitaires et d'intégration du front-end Go depuis le dossier `test/`.

91.2 Responsabilités

- Isoler l'environnement de test Go dans le dossier `test/` conformément à l'architecture du projet.
- Référencer le module principal `sae-git-cli` via la directive `replace sae-git-cli => ../../src/cli`.

---

92 Initialisation

93 Activation des situations de test

Fichier source associé : test/situation_test/init_test.py

93.1 Description

Ce script réactive les éléments masqués pour les tests, en restaurant les fichiers et dossiers Git ou sensibles renommés avec le suffixe `_test`.

93.2 Rôle fonctionnel

Il sert à remettre dans leur état nominal les éléments suivants :

- `.git` → `.git`
- `.gitignore` → `.gitignore`
- `.github` → `.github`
- `.gitattributes` → `.gitattributes`
- `.gitmodules` → `.gitmodules`
- `.env` → `.env`

Le suffixe `_test` est utilisé pour neutraliser les éléments sensibles sans les supprimer du dépôt.

93.3 Comportement

Le script parcourt le dossier racine donné, recherche les entrées portant le suffixe `_test`, puis les renomme vers leur nom original. Il traite d'abord les éléments les plus profondément imbriqués pour éviter les conflits de renommage.

93.4 Utilité dans le projet

Cette étape est nécessaire lorsque les situations de test doivent être exécutées dans un environnement proche du vrai dépôt, notamment pour tester les interactions Git ou les fichiers de configuration sensibles.

---

94 Désinitialisation

95 Neutralisation des situations de test

Fichier source associé : test/situation_test/deinit_test.py

95.1 Description

Ce script masque les éléments Git et sensibles du projet en ajoutant le suffixe `_test` aux fichiers et dossiers concernés. Il s'agit du mécanisme inverse du script d'activation.

95.2 Rôle fonctionnel

Il protège l'environnement de travail en neutralisant automatiquement :

- `.git` → `.git_test`
- `.gitignore` → `.gitignore_test`
- `.github` → `.github_test`
- `.gitattributes` → `.gitattributes_test`
- `.gitmodules` → `.gitmodules_test`
- `.env` → `.env_test`

95.3 Objectif

Cette neutralisation empêche les outils externes, la plateforme de dépôt ou les scanners de sécurité de considérer les dossiers de test comme des ressources réelles du projet.

95.4 Comportement

Le script parcourt les sous-dossiers et renomme les éléments ciblés. Il traite les chemins les plus profonds en premier pour limiter les conflits de renommage lorsque plusieurs éléments sont imbriqués.

95.5 Utilité du projet

C'est une étape de sécurité et de reproducibilité essentielle pour les situations de test interactives, notamment dans des scénarios où le dépôt ou des variables d'environnement ne doivent pas être réellement exploités.

---

96 Déploiement de la documentation

97 Déploiement de la documentation

Cette page explique comment la documentation du projet est construite, publiée et mise à disposition via GitHub Pages.

97.1 Objectif

Le projet utilise MkDocs Material pour générer un site de documentation statique à partir des fichiers Markdown présents dans le dossier `docs/`. Une fois la documentation validée, elle est déployée automatiquement sur GitHub Pages.

97.2 Fichiers concernés

- `mkdocs.yml` : configuration du site MkDocs
- `.github/workflows/ci.yml` (jobs `documentation-build` et `deploy-docs`) : workflow GitHub Actions de build et déploiement
- `docs/` : contenu Markdown de la documentation

97.3 Configuration MkDocs

Le fichier `mkdocs.yml` définit :

- le nom du site : `SAE - Application intelligente`
- le thème : `material`
- la langue : `fr`
- la navigation du site via la clé `nav`

Le build MkDocs transforme les fichiers Markdown en un site statique dans le dossier `site/`.

97.4 Prérequis : activation manuelle de GitHub Pages (une seule fois)

!!! warning “À faire par un administrateur du dépôt” Avant le premier déploiement, GitHub Pages doit être activé manuellement, car le `GITHUB_TOKEN` du workflow n’a pas le droit de créer le site lui-même (erreur `Resource not accessible by integration` dans `actions/configure-pages` sinon) :

```
1. Aller sur **Settings > Pages** du dépôt.
2. Dans **Build and deployment**, mettre **Source = `GitHub Actions`**.
3. Relancer le workflow `CI` (automatique au prochain push sur `main`, ou manuel via **Actions

Le site est ensuite disponible à l'adresse configurée dans `mkdocs.yml` (`site_url`), soit <ht
```

97.5 Prérequis du PDF versionné

!!! warning “Important” Le hook `pre-commit` régénère `documentation.pdf` lorsqu’une page de documentation change. Pandoc et XeLaTeX doivent être installés, sinon le commit est bloqué.

```bash
sudo apt-get update && sudo apt-get install -y pandoc texlive-xetex texlive-
latex-recommended texlive-fonts-recommended lmodern
```

97.6 Workflow de déploiement

Le workflow GitHub Actions est défini dans `.github/workflows/ci.yml` (jobs `documentation-build` puis `deploy-docs`).

97.6.1 Déclenchement

Le build de documentation est exécuté lors de :

- un `push` sur toute branche (`branches: ['**']`)
- un `pull_request` ciblant `main` ou `dev`
- un déclenchement manuel via `workflow_dispatch`

Le déploiement officiel sur GitHub Pages (`deploy-docs` via `actions/deploy-pages@v4`) reste réservé à la branche `main`.

97.6.2 Marqueur de version de développement

Quand la documentation est générée depuis une branche autre que `main`, le titre du site reçoit un suffixe * pour signaler qu’il s’agit d’une version en cours de développement.

- * = documentation en cours de développement
- sans * = documentation officielle de la branche principale

97.6.3 Étapes du job `build`

1. `actions/checkout` : récupération du code source
2. `actions/setup-python` : installation de Python 3.12
3. installation de MkDocs Material
4. ajout du marqueur * pour les branches non principales et détection automatique des nouveaux suivis de sprint (`docs/monitoring/sprint-*.md` ajoutés à la navigation sans modifier `mkdocs.yml`)
5. copie du PDF versionné (`documentation.pdf` à la racine) vers `docs/documentation.pdf` pour exposition sur le site
6. exécution de `mkdocs build`
7. publication d'un artefact de prévisualisation pour validation
8. publication sur GitHub Pages uniquement sur `main`

97.6.4 Étapes du job `deploy`

Le job `deploy` dépend du job `build` et ne se lance que pour `main`. Il publie le contenu sur l'environnement GitHub Pages via :

- `actions/deploy-pages@v4`

Cela permet d'exposer la documentation officielle en ligne, selon l'URL définie par le dépôt GitHub Pages.

97.7 Résultat attendu

Après un push ou un pull request sur `main` ou dev, GitHub Actions reconstruit la documentation. Les branches hors `main` affichent un suffixe * pour indiquer qu'il s'agit d'une version de développement. Seule la branche `main` est déployée publiquement sur GitHub Pages.

97.8 Bonnes pratiques

- garder les fichiers Markdown dans `docs/` cohérents avec la structure du projet
- mettre à jour `mkdocs.yml` si un nouveau chapitre ou une nouvelle page est ajouté
- valider le build localement avec `mkdocs build` avant de pousser

97.9 Commande de vérification locale

Pour construire le site en local :

```
cp documentation.pdf docs/documentation.pdf # expose le lien PDF de l'accueil
mkdocs build
```

Cette commande crée le dossier `site/`, qui est le même contenu que celui déployé sur GitHub Pages.

---

98 Tests automatiques via GitHub Actions

99 Tests automatisés via GitHub Actions

Cette page décrit comment les tests du projet sont lancés automatiquement par les workflows GitHub Actions.

99.1 Objectif

Le projet utilise l'intégration continue pour vérifier automatiquement la qualité du code à chaque pull request et pour exécuter les tests importants dans un environnement standardisé.

99.2 Workflows disponibles

Un seul workflow est présent dans le dossier `.github/workflows/` :

- `ci.yml` : tests Go, tests Python et build/déploiement de la documentation (jobs `go-tests`, `python-tests`, `documentation-build`, `deploy-docs`)

99.3 Workflow CI

Le fichier `.github/workflows/ci.yml` est déclenché sur les `push` (toutes branches), les `pull_request` ciblant `main` et `dev`, et manuellement via `workflow_dispatch`.

99.3.1 Jobs

99.3.1.1 `python-tests` Ce job exécute la suite de tests Python (module C) :

1. installation des dépendances depuis `requirements.txt` (dont `pytest`)
2. exécution de :

```
python -m pytest -q
```

`pytest.ini` exclut par défaut les tests marqués `slow` (ceux qui appellent le vrai serveur Ollama de l'IUT), via `addopts = -m "not slow"`. Le job CI ne lance donc que les tests mockés, qui ne dépendent d'aucune connexion réseau — c'est précisément pour ça que la distinction `slow` / non-`slow` existe.

99.3.1.2 `go-tests` Ce job exécute les tests Go du projet :

1. configuration de Go à partir de `src/cli/go.mod`
2. exécution de :

```
go test ./...
```

Il est lancé depuis le dossier `src/cli` pour vérifier le sous-projet Go.

99.4 Vérification locale

Pour lancer les tests rapides localement :

```
python -m pytest -m "not slow" -q
```

Pour lancer les tests lents :

```
python -m pytest -m slow -q
```

Pour lancer les tests Go :

```
cd src/cli
go test ./...
```

99.5 Intérêt du workflow CI

Le workflow CI sert à :

- détecter les régressions rapidement
- vérifier les modifications avant fusion
- garantir une certaine qualité de code sur les branches principales
- conserver une exécution des tests lourds dans un cadre contrôlé

---

100 Sprint 00

101 Sprint N°1 : Mise en place du projet

User Stories :

- US 1.1 : En tant que développeur, je mets en place un pipeline d'intégration et de déploiement continus (CI/CD)
- US 1.2 : En tant que développeur, je choisis les différentes technologies à utiliser pour le projet (backend / frontend)
- US 1.3 : En tant que développeur, je crée des scénarios de test
- US 1.4 : En tant que développeur, je crée un client pour échanger avec le LLM

---

102 Sprint 01

103 Sprint N°2 : Intégration et développement des fonctionnalités

User Stories :

- US 2.1 : En tant que développeur, je corrige et adapte la récupération des données du `git diff` afin de les rendre compatibles avec le client LLM.
- US 2.2 : En tant que développeur, je mets en place un client Ollama permettant de générer un message de commit à partir des fichiers modifiés.
- US 2.3 : En tant que développeur, je mets en place la validation des réponses générées par le LLM afin de garantir le respect du format attendu.
- US 2.4 : En tant que développeur, je mets en place une gestion des erreurs et un système de nouvelle tentative lors de la génération du message de commit.
- US 2.5 : En tant que développeur, je définis les contrats `DiffFile` et `CommitMessage` permettant d'assurer la communication entre les différents modules.
- US 2.6 : En tant que développeur, je mets en place des tests unitaires et des tests d'intégration pour vérifier le fonctionnement du client LLM.
- US 2.7 : En tant que développeur, je mesure les performances et la conformité des réponses générées par le LLM.

- US 2.8 : En tant que développeur, je mets en place une architecture en miroir du projet afin de faciliter son organisation et sa documentation.
- US 2.9 : En tant que développeur, je réalise un wireframe permettant de définir la structure de l'interface utilisateur.
- US 2.10 : En tant que développeur, je réalise une première version de l'interface utilisateur en Go.

Livrables / DoR & DoD :

---

104 Sprint 02

105 Sprint N°3 : Intégration backend/CLI et fiabilisation du client LLM

User Stories :

- US 3.1 : En tant que développeur, j'écris le point d'entrée `service/core.py` qui relie le CLI Go au backend Python (statut, diff, génération de commit, application du commit, push, release notes, configuration).
- US 3.2 : En tant que développeur, j'unifie la lecture de la configuration (`.env` partagé) afin que tous les membres de l'équipe utilisent la même source de vérité pour l'URL Ollama, le modèle et les timeouts.
- US 3.3 : En tant que développeur, j'ajoute une vérification rapide de la joignabilité du serveur Ollama avant de lancer une génération, pour distinguer un problème réseau (hors réseau IUT) d'une vraie erreur du modèle.
- US 3.4 : En tant que développeur, je corrige la fenêtre de contexte utilisée par Ollama afin d'éviter une troncature silencieuse du diff sur les cas volumineux.
- US 3.5 : En tant que développeur, j'affine le prompt système du client LLM pour réduire les confusions de type Conventional Commits (`fix`/`feat`, `chore`/`build`) observées en mesure.
- US 3.6 : En tant que développeur, je relis le code d'intégration backend/CLI et transmets les anomalies trouvées avant fusion.
- US 3.7 : En tant que développeur, je complète la documentation miroir manquante (tests réseau, script de métriques) pour rester cohérent avec la convention de documentation de l'équipe.
- US 3.8 : En tant que développeur, j'identifie que le déploiement de la documentation utilisateur ne s'exécute jamais réellement, faute de déclencheur CI adapté à notre flux de travail (`Dev` plutôt que `main`).

Livrables / DoR & DoD :

- `service/core.py` fonctionnel, relié au module Git et au module LLM, mergé sur `Dev`.
- Configuration centralisée dans un fichier `.env` partagé, lu par tous les modules Python.
- Client Ollama fiabilisé : ping de disponibilité, fenêtre de contexte explicite, prompt corrigé, revalidé sur des scénarios réels (golden files + fixtures Git réelles).
- Documentation technique à jour pour les nouveaux fichiers de test et de mesure.
- Anomalie de déploiement de la documentation utilisateur identifiée et documentée, correction reportée au sprint suivant.

106 Sprint 03

107 Sprint N°4 : Simulation hors-ligne (Mock IA), robustesse backend et déploiement continu de la documentation

User Stories :

- US 4.1 : En tant que développeur, je développe un module de mock IA (`mock_llm.py`) fournissant des réponses déterministes et statiques (génération de message de commit, release notes, ping de disponibilité) afin de permettre le développement et l'exécution hors-ligne sans dépendre du serveur Ollama de l'IUT.
- US 4.2 : En tant que développeur, j'adapte l'interface CLI Go en remplaçant l'option obsolète `--demo` par deux drapeaux distincts (`--mockInterface` et `--mockIA`), et j'ajoute leur gestion interactive dans le menu de configuration ainsi que la synchronisation de la variable `MOCK_AI` dans le fichier `.env`.
- US 4.3 : En tant que développeur, je sécurise le point d'entrée `service/core.py` en évitant l'arrêt brutal par `sys.exit(1)` sur action inconnue au profit d'un retour JSON exploitable par le CLI Go, et j'exploite la méthode publique `get_staged_diff` du wrapper Git.
- US 4.4 : En tant que développeur, j'écris une suite de tests unitaires complète pour le mock IA (`test_mock_ai.py`) isolée par `conftest.py`, ainsi qu'un test unitaire Go pour la persistance conjointe des options de mock (`TestSaveConfigBothMocks`).
- US 4.5 : En tant que développeur, je résous l'anomalie de déploiement de la documentation identifiée au sprint précédent en automatisant la publication du site MkDocs sur GitHub Pages lors des pushs et pull requests vers `Dev` et `main`.
- US 4.6 : En tant que développeur, je mets en place la génération automatisée d'un livrable PDF de documentation (`documentation.pdf`) via un script dédié (`generate_documentation_pdf.py`), un hook git de pré-commit et son intégration dans le workflow CI.
- US 4.7 : En tant que développeur, j'intègre l'exécution automatique des tests Go dans le pipeline CI (`.github/workflows/ci.yml`) à chaque push et pull request.
- US 4.8 : En tant que développeur, j'actualise l'ensemble de la documentation technique miroir (`mock_llm.md`, `test_mock_ai.md`, `llm.md`, `deploiement_documentation.md`) et les guides utilisateurs pour refléter les évolutions architecturales et fonctionnelles du projet.
- US 4.9 : En tant que développeur, je restaure le job de tests unitaires Python (`python-tests`) dans le workflow CI (`.github/workflows/ci.yml`), en ajoutant la dépendance `pytest>=7.0.0` dans `requirements.txt` et en actualisant `docs/deploiement_tests_workflows.md` pour exécuter automatiquement les 40 tests unitaires mockés sans nécessiter de connexion au réseau IUT.

Livrables / DoR & DoD :

- Module `MockLLMClient` (`src/git_commit_release_notes_generator/ollama_client/mock_llm.py`) fonctionnel et intégré, activable via la variable `MOCK_AI` du `.env`.

- CLI Go mis à niveau avec prise en charge des flags `--mockInterface` et `--mockIA`, affichage des statuts dans le menu principal et gestion dans le menu de configuration.
- Point d'entrée `service/core.py` fiabilisé : retours d'erreurs en JSON sans arrêt non contrôlé du sous-processus et harmonisation des appels au wrapper Git (`get_staged_diff`).
- Job CI `python-tests` restauré dans `.github/workflows/ci.yml` (avec `pytest>=7.0.0` dans `requirements.txt` et mise à jour de `docs/deploiement_tests_workflows.md`), exécutant les 40 tests unitaires Python hors-ligne en parallèle des tests Go (`bridge_test.go`).
- Déploiement automatique du site de documentation MkDocs sur GitHub Pages opérationnel sur les branches `Dev` et `main` (anomalie du Sprint N°3 résolue).
- Génération automatisée du livrable `documentation.pdf` intégrée à la CI et outillée localement avec un hook Git pre-commit.
- Documentation technique miroir et guides utilisateurs synchronisés avec l'arborescence du projet.
- Pistes d'amélioration UX (verrouillage de validation en cas d'erreur de génération, notifications de fin de traitement de l'IA) identifiées sur le Trello et planifiées pour le sprint suivant.

---

108 Sprint 04

109 Sprint N°5 : Diagnostic réseau unifié, stabilisation CI/CD, version stable sur main et améliorations UX

User Stories :

- US 5.1 : En tant que développeur, j'unifie la vérification de connectivité avec le serveur Ollama via une fonction centralisée `check_ollama_reachable` (alias `ping_ollama`) et une structure enrichie `PingResult` dans `ollama_client/llm.py`, fournissant la latence réseau en millisecondes, le statut d'accessibilité, la liste des modèles installés et une compatibilité de déballage en tuple.
- US 5.2 : En tant que développeur, je sécurise le point d'entrée `service/core.py` en intégrant ce ping unifié avant toute requête et en appliquant un mécanisme de fail-fast strict sur `action_release_notes` afin de renvoyer une erreur explicite au CLI Go plutôt que d'utiliser silencieusement un modèle déterministe lors d'une rupture de connexion.
- US 5.3 : En tant que développeur, j'expose explicitement la variable `OLLAMA_NUM_CTX=32768` dans le fichier `.env.exemple` partagé pour aligner les environnements sur la fenêtre de contexte maximale du modèle et prévenir la troncature silencieuse des diffs volumineux.
- US 5.4 : En tant que développeur, j'écris une suite de tests unitaires pour le diagnostic Ollama (`test/test_ollama_llm.py` et `test/test_core.py`), couvrant la mesure de latence, l'extraction des modèles, la compatibilité d'interface et l'interruption précoce sur indisponibilité réseau, portant la suite automatisée à 45 tests unitaires passés.
- US 5.5 : En tant que développeur, je stabilise et sécurise le pipeline CI GitHub Actions (`.github/workflows/ci.yml`) en affinant les déclencheurs de pull request et de push vers dev et `main`, en restreignant les permissions par défaut, et en épinglant les dépendances MkDocs (`mkdocs==1.6.0`, `mkdocs-material==9.5.34`, `PyYAML==6.0.2`) pour garantir des builds déterministes.
- US 5.6 : En tant que développeur, je résous les conflits de nommage sur le backend en supprimant `src/logging.py` qui masquait la bibliothèque standard Python, j'assainis le

dépôt Git (`.gitignore`, suppression de fichiers vides) et j'automatise l'activation et la publication de GitHub Pages via l'API d'enablement GitHub.

- US 5.7 : En tant que développeur, je déploie la première version stable officielle de l'application sur la branche `main` (PR #36 et PR #37) après validation conjointe de tous les contrôles qualité CI (tests Go, tests Python mockés, génération du PDF et site MkDocs).
- US 5.8 : En tant que développeur, je conçois un composant standardisé d'affichage des cartes d'erreur (`PrintErrorCard`) dans `src/cli/ui/` afin de supprimer le contournement technique de `commit_view.go` (injection d'un faux commit de type `"erreur"`) et harmoniser la restitution des erreurs dans l'ensemble des vues de l'interface Go.
- US 5.9 : En tant que développeur, je renforce les contrôles de flux de travail en verrouillant les choix de validation du CLI Go en cas d'erreur de génération et en préparant une boucle de nouvel essai (retry loop) pour rattraper les non-conformités de formatage de l'IA.
- US 5.10 : En tant que développeur, j'améliore l'expérience utilisateur (UX) du CLI Go en intégrant des indicateurs ou notifications de fin de traitement lors des requêtes d'analyse du modèle afin d'avertir clairement l'utilisateur dès que la génération est terminée.
- US 5.11 : En tant que développeur, je prépare la réalisation du poster demandé par le client afin de synthétiser les fonctionnalités, l'architecture technique et les bénéfices de l'application en vue de la soutenance.

Livrables / DoR & DoD :

- Système de diagnostic et de ping réseau Ollama unifié (`PingResult`, `check_ollama_reachable`, `ping_ollama`) opérationnel, testé unitairement (45 tests unitaires Python validés hors-ligne) et documenté dans les fiches miroirs (`llm.md`, `core.md`, `test_core.md`, `test_ollama_llm.md`).
- Comportement fail-fast strict déployé sur la génération des release notes dans `service/core.py` en cas d'indisponibilité du serveur IA.
- Variable d'environnement `OLLAMA_NUM_CTX` officialisée et documentée dans `.env.exemple`.
- Pipeline CI/CD GitHub Actions fiabilisé, allégé et reproductible : exécution parallèle des tests Go et Python déconnectés, permissions restreintes, épinglage des dépendances MkDocs et déploiement continu GitHub Pages sans blocage.
- Dépôt nettoyé et assaini : suppression du conflit stdlib `src/logging.py`, nettoyage des fichiers vides et réconciliation du `.gitignore`.
- Première version stable officielle validée et livrée sur la branche `main` (PR #36 & PR #37).
- Spécification et architecture du composant d'erreur TUI unifié (`PrintErrorCard`) et des garde-fous de validation définies pour l'interface CLI Go.
- Préparation et maquettage du poster de présentation demandé par le client pour la soutenance finale.
- Suivi de projet synchronisé sur la branche `sprints` et aligné avec les exigences du jalon du 16 octobre.
